NAME : Prathiksha S
REG NO : 241801210

PRINCIPLES OF DATA SCIENCE

LAB EXPERIMENTS

EXPERIMENT 1: TELCO CUSTOMER CHURN DATASET

AIM

To inspect and analyze the Telco Customer Churn dataset using Pandas and basic data-preparation operations.

QUESTION / TASK

Load Telco-Customer-Churn.csv. Display the first and last records, dataset shape and column names,

CODE AND OUTPUT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
from google.colab import files
uploaded = files.upload()

df = pd.read_csv("Telco-Customer-Churn.csv")
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving Telco-Customer-Churn.csv to Telco-Customer-Churn (1).csv

```
print("First 10 Records")
print(df.head(10))
```

```
First 10 Records
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes
5  9305-CDSKC  Female              0      No         No       8          Yes
6  1452-KIOVK    Male              0      No        Yes      22          Yes
7  6713-OKOMC  Female              0      No         No      10           No
8  7892-POOKP  Female              0     Yes         No      28          Yes
9  6388-TABGU    Male              0      No        Yes      62          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No
5               Yes     Fiber optic             No  ...              Yes
6               Yes     Fiber optic             No  ...               No
7  No phone service             DSL            Yes  ...               No
8               Yes     Fiber optic             No  ...              Yes
9                No             DSL            Yes  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes
5          No         Yes             Yes  Month-to-month              Yes
6          No         Yes              No  Month-to-month              Yes
7          No          No              No  Month-to-month               No
8         Yes         Yes             Yes  Month-to-month              Yes
9          No          No              No        One year               No

              PaymentMethod MonthlyCharges  TotalCharges Churn
0          Electronic check          29.85         29.85    No
1              Mailed check          56.95        1889.5    No
2              Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4          Electronic check          70.70        151.65   Yes
5          Electronic check          99.65         820.5   Yes
6   Credit card (automatic)          89.10        1949.4    No
7              Mailed check          29.75         301.9    No
8          Electronic check         104.80       3046.05   Yes
9  Bank transfer (automatic)          56.15       3487.95    No

[10 rows x 21 columns]
```

```
print("\nLast 10 Records")
print(df.tail(10))
```

```
Last 10 Records
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
7033  9767-FFLEM    Male              0      No         No      38
7034  0639-TSIQW  Female              0      No         No      67
```

```
7035  8456-QDAVC    Male              0      No         No      19
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ...  \
7033          Yes                No     Fiber optic                   No  ...
7034          Yes               Yes     Fiber optic                  Yes  ...
7035          Yes                No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
7037          Yes                No              No  No internet service  ...
7038          Yes               Yes             DSL                  Yes  ...
7039          Yes               Yes     Fiber optic                   No  ...
7040           No  No phone service             DSL                  Yes  ...
7041          Yes               Yes     Fiber optic                   No  ...
7042          Yes                No     Fiber optic                  Yes  ...

         DeviceProtection          TechSupport          StreamingTV  \
7033                   No                   No                  No
7034                  Yes                   No                 Yes
7035                   No                   No                 Yes
7036                  Yes                  Yes                 Yes
7037  No internet service  No internet service  No internet service
7038                  Yes                  Yes                 Yes
7039                  Yes                   No                 Yes
7040                   No                   No                  No
7041                   No                   No                  No
7042                  Yes                  Yes                 Yes

         StreamingMovies        Contract PaperlessBilling  \
7033                  No  Month-to-month              Yes
7034                  No  Month-to-month              Yes
7035                  No  Month-to-month              Yes
7036                 Yes        One year               No
7037  No internet service        Two year              Yes
7038                 Yes        One year              Yes
7039                 Yes        One year              Yes
7040                  No  Month-to-month              Yes
7041                  No  Month-to-month              Yes
7042                 Yes        Two year              Yes

                 PaymentMethod MonthlyCharges  TotalCharges Churn
7033    Credit card (automatic)          69.50       2625.25    No
7034    Credit card (automatic)         102.95       6886.25   Yes
7035  Bank transfer (automatic)          78.70       1495.1    No
7036           Electronic check          60.65        743.3    No
7037  Bank transfer (automatic)          21.15       1419.4    No
7038               Mailed check          84.80       1990.5    No
7039    Credit card (automatic)         103.20       7362.9    No
```

```
print("\nShape of Dataset:", df.shape)
print("\nColumn Names:")
print(df.columns)
```

```
Shape of Dataset: (7043, 21)

Column Names:
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

```
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
```

```
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None
       SeniorCitizen       tenure  MonthlyCharges
count    7043.000000  7043.000000     7043.000000
mean        0.162147    32.371149       64.761692
std         0.368612    24.559481       30.090047
min         0.000000     0.000000       18.250000
25%         0.000000     9.000000       35.500000
50%         0.000000    29.000000       70.350000
75%         0.000000    55.000000       89.850000
max         1.000000    72.000000      118.750000
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
```

```
df = df.drop_duplicates()
if 'TotalCharges' in df.columns:
    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
numerical_cols = df.select_dtypes(include=np.number).columns
categorical_cols = df.select_dtypes(exclude=np.number).columns
for col in numerical_cols:
    df[col] = df[col].fillna(df[col].median())
for col in categorical_cols:
    df[col] = df[col].fillna(df[col].mode()[0])
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())
```

```
Missing Values After Cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
```

```
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
columns_to_encode = [
    'gender',
    'Partner',
    'Dependents',
    'Contract',
    'PaymentMethod',
    'InternetService',
    'Churn'
]
le = LabelEncoder()

for col in columns_to_encode:
    if col in df.columns:
        df[col] = le.fit_transform(df[col])
print("\nEncoded Dataset Sample:")
print(df.head())
```

```
Encoded Dataset Sample:
   customerID  gender  SeniorCitizen  Partner  Dependents  tenure  \
0  7590-VHVEG       0              0        1           0       1
1  5575-GNVDE       1              0        0           0      34
2  3668-QPYBK       1              0        0           0       2
3  7795-CFOCW       1              0        0           0      45
4  9237-HQITU       0              0        0           0       2

  PhoneService     MultipleLines  InternetService OnlineSecurity  ...  \
0           No  No phone service                0             No  ...
1          Yes                No                0            Yes  ...
2          Yes                No                0            Yes  ...
3           No  No phone service                0            Yes  ...
4          Yes                No                1             No  ...

  DeviceProtection TechSupport StreamingTV StreamingMovies Contract  \
0               No          No          No              No        0
1              Yes          No          No              No        1
2               No          No          No              No        0
3              Yes         Yes          No              No        1
4               No          No          No              No        0

   PaperlessBilling PaymentMethod  MonthlyCharges  TotalCharges  Churn
0               Yes             2           29.85         29.85      0
1                No             3           56.95       1889.50      0
2               Yes             3           53.85        108.15      1
3                No             0           42.30       1840.75      0
4               Yes             2           70.70        151.65      1

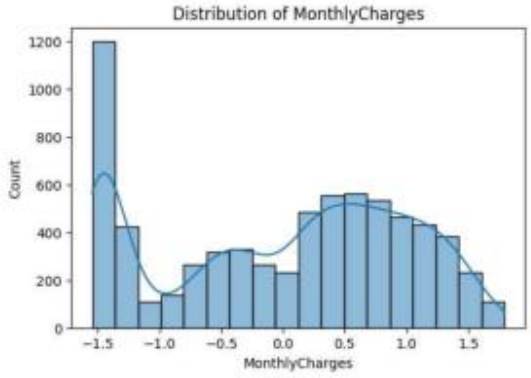
[5 rows x 21 columns]
```

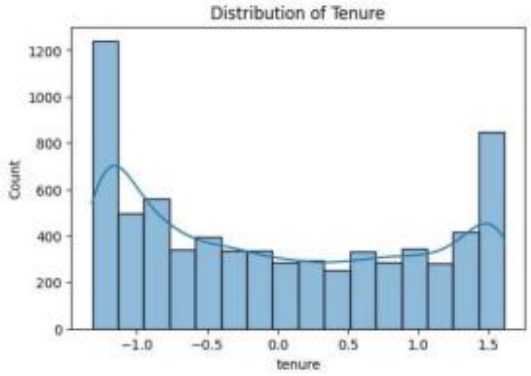
```
scaler = StandardScaler()
features_to_scale = [
    'tenure',
    'MonthlyCharges',
    'TotalCharges'
]
available_features = [col for col in features_to_scale if col in df.columns]
df[available_features] = scaler.fit_transform(df[available_features])
print("\nScaled Numerical Features:")
print(df[available_features].head())
```

```
Scaled Numerical Features:
     tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1  0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3  0.514251       -0.746535     -0.194766
4 -1.236724        0.197365     -0.940470
```

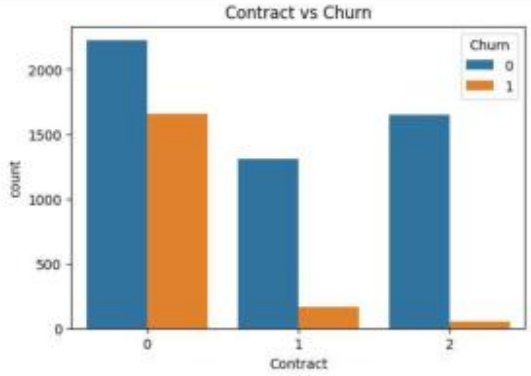
```
plt.figure(figsize=(6,4))
sns.histplot(df['MonthlyCharges'], kde=True)
plt.title("Distribution of MonthlyCharges")
plt.show()
plt.figure(figsize=(6,4))
```

```
sns.histplot(df['tenure'], kde=True)
plt.title("Distribution of Tenure")
plt.show()
```



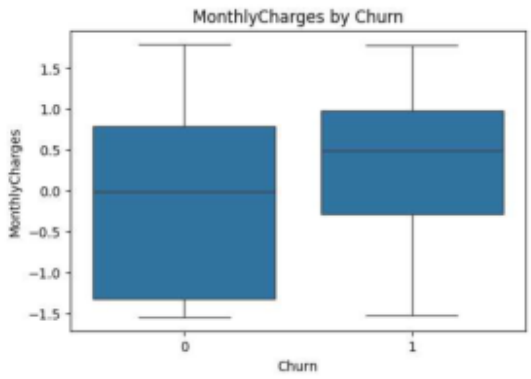



```
if 'Contract' in df.columns and 'Churn' in df.columns:
    plt.figure(figsize=(6,4))
    sns.countplot(x='Contract', hue='Churn', data=df)
    plt.title("Contract vs Churn")
    plt.show()
```



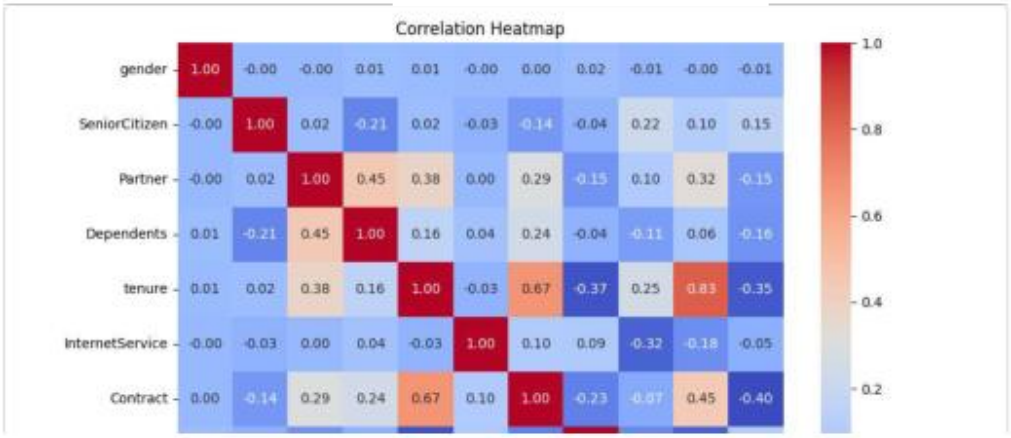

```
if 'MonthlyCharges' in df.columns and 'Churn' in df.columns:
    plt.figure(figsize=(6,4))
```

```
sns.boxplot(x='Churn', y='MonthlyCharges', data=df)
plt.title("MonthlyCharges by Churn")
plt.show()
```




```
plt.figure(figsize=(10,8))
corr = df.select_dtypes(include=np.number).corr()

sns.heatmap(
    corr,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)
plt.title("Correlation Heatmap")
plt.show()
```



RESULT

The Telco Customer Churn dataset was loaded. Its records, dimensions, columns, data types, descriptive statistics, missing values, and duplicate records were examined.

EXPERIMENT 2: TITANIC DATASET ANALYSIS

AIM

To explore the Titanic dataset and perform basic data cleaning and passenger-data analysis.

QUESTION / TASK

Load Titanic-Dataset.csv. Display the dataset shape, information, and descriptive statistics; handle missing Age, Cabin, and Embarked values; plot passenger-class distribution; and select passengers using the conditions shown in the Colab code.

CODE AND OUTPUT


```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
```

```
df = pd.read_csv("Titanic-Dataset.csv")
```

```
print(df.shape)
print(df.info())
print(df.describe())
```

```
(891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
None
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='mean')
df['Age'] = imputer.fit_transform(df[['Age']])

print(df['Age'].isnull().sum())
```

```
0
```

```
df['Cabin'] = df['Cabin'].fillna("Unknown")

print(df['Cabin'].isnull().sum())
```

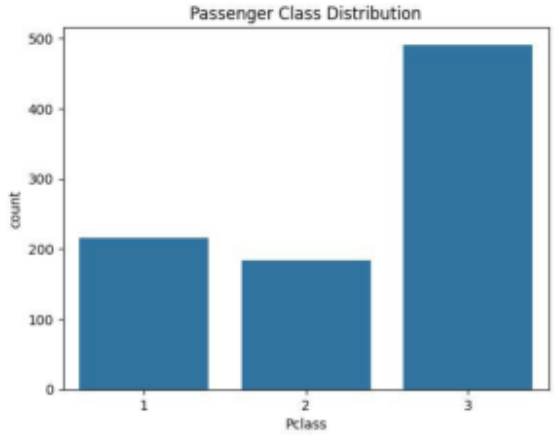
```
0
```

```
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
print(df['Embarked'].isnull().sum())
```

```
0
```

```
import seaborn as sns
sns.countplot(x='Pclass', data=df)
```

```
plt.title("Passenger Class Distribution")
plt.show()
```




```
female_survived = df[(df['Sex'] == 'female') & (df['Survived'] == 1)]

print(female_survived['Name'])
```

```
1      Cumings, Mrs. John Bradley (Florence Briggs Th...
2                                  Heikkinen, Miss. Laina
3           Futrelle, Mrs. Jacques Heath (Lily May Peel)
8      Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9                    Nasser, Mrs. Nicholas (Adele Achem)
                             ...
874                Abelson, Mrs. Samuel (Hannah Wizosky)
875                    Najib, Miss. Adele Kiamie "Jane"
879        Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)
880         Shelley, Mrs. William (Imanita Parrish Hall)
887                          Graham, Miss. Margaret Edith
Name: Name, Length: 233, dtype: object
```

```
result = df[(df['Pclass'] == 3) & (df['Age'] < 18)]

print(result[['Name', 'Age', 'Pclass']])
```

```
                                     Name   Age  Pclass
7               Palsson, Master. Gosta Leonard   2.0       3
10            Sandstrom, Miss. Marguerite Rut   4.0       3
14       Vestrom, Miss. Hulda Amanda Adolfina  14.0       3
16                       Rice, Master. Eugene   2.0       3
22                McGowan, Miss. Anna "Annie"  15.0       3
..                                        ...   ...     ...
844                      Culumovic, Mr. Jeso  17.0       3
850  Andersson, Master. Sigvard Harald Elias   4.0       3
852                   Boulos, Miss. Nourelain   9.0       3
869           Johnson, Master. Harold Theodor   4.0       3
875          Najib, Miss. Adele Kiamie "Jane"  15.0       3

[78 rows x 3 columns]
```

```
result = df[(df['Pclass'] == 1) & (df['Age'] > 40)]

print(result[['Name', 'Age', 'Pclass']])
```

```
                                              Name   Age  Pclass
6                            McCarthy, Mr. Timothy J  54.0       1
11                          Bonnell, Miss. Elizabeth  58.0       1
35                       Holverson, Mr. Alexander Oskar  42.0       1
52             Harper, Mrs. Henry Sleeper (Myna Haxtun)  49.0       1
54                       Ostby, Mr. Engelhart Cornelius  65.0       1
..                                                  ...   ...     ...
856         Wick, Mrs. George Dennick (Mary Hitchcock)  45.0       1
857                               Daly, Mr. Peter Denis  51.0       1
862  Swift, Mrs. Frederick Joel (Margaret Welles Ba...  48.0       1
871   Beckwith, Mrs. Richard Leonard (Sallie Monypeny)  47.0       1
```

```
879      Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)  56.0       1

[76 rows x 3 columns]
```

```
result = df[(df['Pclass'] == 1) &
            (df['Age'] > 40) &
            (df['Survived'] == 1)]

print(result[['Name', 'Age']])
```

```
                                                  Name   Age
11                            Bonnell, Miss. Elizabeth  58.0
52           Harper, Mrs. Henry Sleeper (Myna Haxtun)  49.0
187      Romaine, Mr. Charles Hallace ("Mr C Rolmane")  45.0
194          Brown, Mrs. James Joseph (Margaret Tobin)  44.0
195                                 Lurette, Miss. Elise  58.0
268      Graham, Mrs. William Thompson (Edith Junkins)  58.0
275                  Andrews, Miss. Kornelia Theodosia  63.0
299    Baxter, Mrs. James (Helene DeLaudeniere Chaput)  50.0
337                     Burns, Miss. Elizabeth Margaret  41.0
366  Warren, Mrs. Frank Manley (Anna Sophia Atkinson)  60.0
380                               Bidois, Miss. Rosalie  42.0
449                     Peuchen, Major. Arthur Godfrey  52.0
453                           Goldenberg, Mr. Samuel L  49.0
460                                Anderson, Mr. Harry  48.0
496                      Eustis, Miss. Elizabeth Mussey  54.0
513    Rothschild, Mrs. Martin (Elizabeth L. Barrett)  54.0
523   Hippach, Mrs. Louis Albert (Ida Sophia Fischer)  44.0
556  Duff Gordon, Lady. (Lucille Christiana Sutherl...  48.0
571      Appleton, Mrs. Edward Dale (Charlotte Lamson)  53.0
587                  Frolicher-Stehli, Mr. Maxmillian  60.0
591   Stephenson, Mrs. Walter Bertram (Martha Eustis)  52.0
599      Duff Gordon, Sir. Cosmo Edmund ("Mr Morgan")  49.0
621                     Kimball, Mr. Edwin Nelson Jr  42.0
630              Barkworth, Mr. Algernon Henry Wilson  80.0
645                          Harper, Mr. Henry Sleeper  48.0
647               Simonius-Blumer, Col. Oberst Alfons  56.0
660                    Frauenthal, Dr. Henry William  50.0
707                 Calderhead, Mr. Edward Pennington  42.0
712                          Taylor, Mr. Elmer Zebley  48.0
765                Hogeboom, Mrs. John C (Anna Andrews)  51.0
779  Robert, Mrs. Edward Scott (Elisabeth Walton Mc...  43.0
796                      Leader, Dr. Alice (Farnham)  49.0
820  Hays, Mrs. Charles Melville (Clara Jennings Gr...  52.0
829         Stone, Mrs. George Nelson (Martha Evelyn)  62.0
856        Wick, Mrs. George Dennick (Mary Hitchcock)  45.0
857                            Daly, Mr. Peter Denis   51.0
862  Swift, Mrs. Frederick Joel (Margaret Welles Ba...  48.0
871   Beckwith, Mrs. Richard Leonard (Sallie Monypeny)  47.0
879      Potter, Mrs. Thomas Jr (Lily Alexenia Wilson)  56.0
```

```
result = df[(df['Sex'] == 'male') & (df['Fare'] > 100)]

print(result[['Name', 'Fare']])
```

```
                                        Name      Fare
27              Fortune, Mr. Charles Alexander  263.0000
118                  Baxter, Mr. Quigg Edmond  247.5208
305             Allison, Master. Hudson Trevor  151.5500
332                  Graham, Mr. George Edward  153.4625
373                      Ringhini, Mr. Sante  135.6333
377                  Widener, Mr. Harry Elkins  211.5000
390                 Carter, Mr. William Ernest  120.0000
438                          Fortune, Mr. Mark  263.0000
505  Penasco y Castellana, Mr. Victor de Satode  108.9000
527                        Farthing, Mr. John  221.7792
544                 Douglas, Mr. Walter Donald  106.4250
550                Thayer, Mr. John Borland Jr  110.8833
557                       Robbins, Mr. Victor  227.5250
659                 Newell, Mr. Arthur Webster  113.2750
660              Frauenthal, Dr. Henry William  133.6500
679         Cardeza, Mr. Thomas Drake Martinez  512.3292
698                   Thayer, Mr. John Borland  110.8833
737                     Lesurer, Mr. Gustave J  512.3292
802        Carter, Master. William Thornton II  120.0000
```

```
result = df[(df['Embarked'] == 'C') & (df['Pclass'] == 2)]

print(result[['Name', 'Embarked', 'Pclass']])
```

```
                                                 Name Embarked  Pclass
9                   Nasser, Mrs. Nicholas (Adele Achem)        C       2
43            Laroche, Miss. Simonne Marie Anne Andree        C       2
122                                 Nasser, Mr. Nicholas        C       2
135                                   Richard, Mr. Emile        C       2
181                                     Pernot, Mr. Rene        C       2
292                               Levy, Mr. Rene Jacques        C       2
308                                  Abelson, Mr. Samuel        C       2
361                             del Carlo, Mr. Sebastiano        C       2
389                                  Lehmann, Miss. Bertha        C       2
473       Jerwan, Mrs. Amin S (Marie Marthe Thuillard)        C       2
547                             Padro y Manent, Mr. Julian        C       2
608  Laroche, Mrs. Joseph (Juliette Marie Louise La...        C       2
685              Laroche, Mr. Joseph Philippe Lemercier        C       2
817                                   Mallet, Mr. Albert        C       2
827                                 Mallet, Master. Andre        C       2
866                          Duran y More, Miss. Asuncion        C       2
874               Abelson, Mrs. Samuel (Hannah Wizosky)        C       2
```

```
result = df[(df['Embarked'] == 'C') & (df['Pclass'] == 2)]

print(result[['Name', 'Embarked', 'Pclass']])
```

```
                                                 Name Embarked  Pclass
9                   Nasser, Mrs. Nicholas (Adele Achem)        C       2
43            Laroche, Miss. Simonne Marie Anne Andree        C       2
122                                 Nasser, Mr. Nicholas        C       2
135                                   Richard, Mr. Emile        C       2
181                                     Pernot, Mr. Rene        C       2
292                               Levy, Mr. Rene Jacques        C       2
308                                  Abelson, Mr. Samuel        C       2
361                             del Carlo, Mr. Sebastiano        C       2
389                                  Lehmann, Miss. Bertha        C       2
473       Jerwan, Mrs. Amin S (Marie Marthe Thuillard)        C       2
547                             Padro y Manent, Mr. Julian        C       2
608  Laroche, Mrs. Joseph (Juliette Marie Louise La...        C       2
685              Laroche, Mr. Joseph Philippe Lemercier        C       2
817                                   Mallet, Mr. Albert        C       2
827                                 Mallet, Master. Andre        C       2
866                          Duran y More, Miss. Asuncion        C       2
874               Abelson, Mrs. Samuel (Hannah Wizosky)        C       2
```

```
result = df[(df['Survived'] == 0) &
            (df['SibSp'] == 0) &
            (df['Parch'] == 0)]

print(result[['Name', 'Survived']])
```

```
                                     Name  Survived
4                  Allen, Mr. William Henry         0
5                          Moran, Mr. James         0
6                   McCarthy, Mr. Timothy J         0
12           Saundercock, Mr. William Henry         0
14   Vestrom, Miss. Hulda Amanda Adolfina         0
..                                    ...       ...
882            Dahlberg, Miss. Gerda Ulrika         0
883          Banfield, Mr. Frederick James         0
884                 Sutehall, Mr. Henry Jr         0
886                  Montvila, Rev. Juozas         0
890                    Dooley, Mr. Patrick         0

[374 rows x 2 columns]
```

```
result = df[df['Survived'] == 1]

top5 = result.sort_values(by='Age', ascending=False).head(5)

print(top5[['Name', 'Age']])
```

```
                                        Name   Age
630       Barkworth, Mr. Algernon Henry Wilson  80.0
275          Andrews, Miss. Kornelia Theodosia  63.0
483                     Turkula, Mrs. (Hedwig)  63.0
570                        Harris, Mr. George  62.0
829  Stone, Mrs. George Nelson (Martha Evelyn)  62.0
```

```
result = df[df['Fare'] == 0]

print(result[['Name', 'Fare']])
```

	Name	Fare
179	Leonard, Mr. Lionel	0.0
263	Harrison, Mr. William	0.0
271	Tornquist, Mr. William Henry	0.0
277	Parkes, Mr. Francis "Frank"	0.0
302	Johnson, Mr. William Cahoone Jr	0.0
413	Cunningham, Mr. Alfred Fleming	0.0
466	Campbell, Mr. William	0.0
481	Frost, Mr. Anthony Wood "Archie"	0.0
597	Johnson, Mr. Alfred	0.0
633	Parr, Mr. William Henry Marsh	0.0
674	Watson, Mr. Ennis Hastings	0.0
732	Knight, Mr. Robert J	0.0
806	Andrews, Mr. Thomas Jr	0.0
815	Fry, Mr. Richard	0.0
822	Reuchlin, Jonkheer. John George	0.0

```
from sklearn.model_selection import train_test_split

X = df.drop('Survived', axis=1)
y = df['Survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (712, 11)
Testing Data Shape: (179, 11)
```


RESULT

The dataset summary was generated, missing values in the selected columns were handled, passenger-class distribution was plotted, and the specified passenger groups were filtered.

EXPERIMENT 3(a): ADVERTISING DATA — SALES PREDICTION

AIM

To explore the Advertising dataset and use regression models to predict Sales from advertising features.

QUESTION / TASK

Load Advertising.csv, inspect missing values and data types, visualize the TV, Radio, and Newspaper features, split the data into training and testing sets, and apply the regression models shown in the Colab notebook.

CODE AND OUTPUT


```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.svm import SVR
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
```

```
from google.colab import files
uploaded = files.upload()
```

```
Choose Files  No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Advertising.csv to Advertising.csv
```

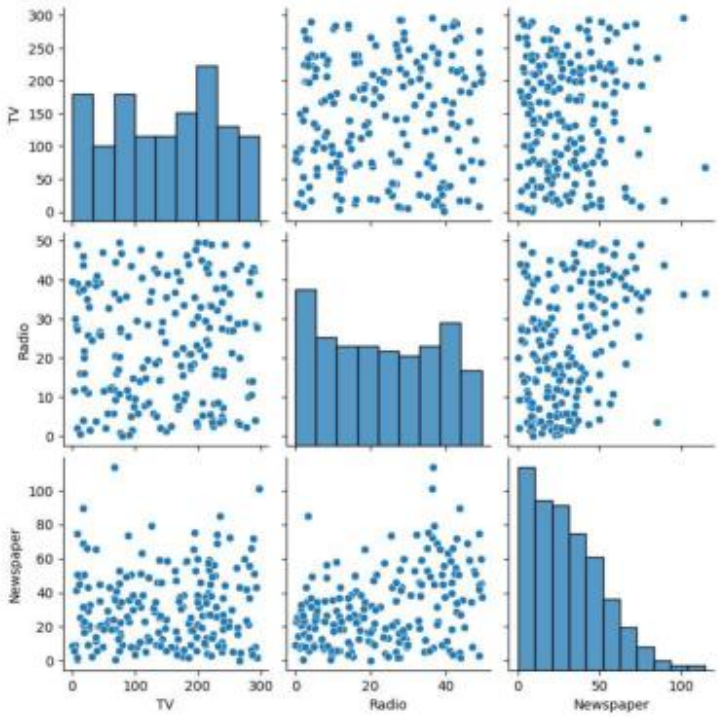
```
df = pd.read_csv("Advertising.csv")
print(df.head())
```

```
   Unnamed: 0     TV  Radio  Newspaper  Sales
0           1  230.1   37.8       69.2   22.1
1           2   44.5   39.3       45.1   10.4
2           3   17.2   45.9       69.3    9.3
3           4  151.5   41.3       58.5   18.5
4           5  180.8   10.8       58.4   12.9
```

```
print(df.isnull().sum())
print(df.dtypes)
```

```
Unnamed: 0    0
TV            0
Radio         0
Newspaper     0
Sales         0
dtype: int64
Unnamed: 0      int64
TV            float64
Radio         float64
Newspaper     float64
Sales         float64
dtype: object
```

```
X = df[['TV', 'Radio', 'Newspaper']]
y = df['Sales']
sns.pairplot(X)
plt.show()
```




```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
print(X_train.shape)
print(X_test.shape)
```

```
(160, 3)
(40, 3)
```

```
from sklearn.linear_model import LinearRegression
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)
print("Predicted Sales:")
print(y_pred_linear)
```

```
Predicted Sales:
[16.4080242  20.88988289 21.55384318 10.60850256 22.11237326 13.10559172
 21.05719192  7.46101034 13.60634581 15.15506967  9.04831992  6.65328312
 14.34554487  8.90349333  9.68950028 12.16494386  8.73628397 16.26507258
 10.27759582 18.83109103 19.56036653 13.25103464 12.33620695 21.30695132
  7.82740385  5.80957448 20.75753231 11.98138077  9.18349576  8.5066991
 12.46646769 10.00337695 21.3876709  12.24966368 18.26661538 20.13766267
 14.05514005 20.85411186 11.0174441   4.56899622]
```

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
poly_model = Pipeline([
    ('poly', PolynomialFeatures(degree=2)),
    ('linear', LinearRegression())
])
poly_model.fit(X_train, y_train)
```

```
y_pred_poly = poly_model.predict(X_test)
print("Polynomial Regression Predictions:")
print(y_pred_poly)
```

```
Polynomial Regression Predictions:
[17.25443578 22.7193321  20.42799436  7.542709   24.37103037 12.55927164
 22.80784986  8.34371358 12.0586023  15.69024565  7.89166367  8.27082715
 11.86055971  6.16501094 10.57664393 12.3628954   6.74390963 16.65082111
 10.68661722 19.03888161 20.15223945 13.1137814   9.56273868 22.10675225
  8.96725241  7.7794437  22.48745151 12.72865973 10.25529986  6.22368636
 11.64677688 10.22431946 23.39763949  9.17403232 15.36143449 21.05675814
 10.9795286  20.23370753 11.85153256  6.58779915]
```

```
from sklearn.svm import SVR

svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

print("SVR Predictions:")
print(y_pred_svr)
```

```
SVR Predictions:
[16.82433682 19.737857   18.51883217  8.49757937 19.14449648 12.35892516
 19.35370785  8.670194   14.26079367 15.69961405  8.67459241  8.63883765
 16.10338892  7.86941414 11.80945227 13.73449615  7.82912158 16.07178231
 10.8096626  17.39870345 19.02145965 12.85124102  9.98938778 19.67786747
  9.38640699  8.22363571 19.35662438 12.9459954  10.6084116   7.91787133
 13.69013745 10.3362837  20.05661258  9.88723936 17.12640815 17.96277376
 11.10345511 18.94964584 12.02692034  7.21381614]
```

```
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae_linear = mean_absolute_error(y_test, y_pred_linear)
mse_linear = mean_squared_error(y_test, y_pred_linear)
rmse_linear = np.sqrt(mse_linear)

mae_poly = mean_absolute_error(y_test, y_pred_poly)
mse_poly = mean_squared_error(y_test, y_pred_poly)
rmse_poly = np.sqrt(mse_poly)

mae_svr = mean_absolute_error(y_test, y_pred_svr)
mse_svr = mean_squared_error(y_test, y_pred_svr)
rmse_svr = np.sqrt(mse_svr)

print("Linear Regression")
print("MAE  :", mae_linear)
print("MSE  :", mse_linear)
print("RMSE :", rmse_linear)

print("\nPolynomial Regression")
print("MAE  :", mae_poly)
print("MSE  :", mse_poly)
print("RMSE :", rmse_poly)

print("\nSVR")
print("MAE  :", mae_svr)
print("MSE  :", mse_svr)
print("RMSE :", rmse_svr)
```

```
Linear Regression
MAE  : 1.4607567168117603
MSE  : 3.1740973539761033
RMSE : 1.78159966153345

Polynomial Regression
MAE  : 0.5261794444043838
MSE  : 0.41291022853790765
RMSE : 0.6425809120553673

SVR
MAE  : 1.514357586594181
MSE  : 4.00604540443217
RMSE : 2.0015107804936174
```

```
import matplotlib.pyplot as plt

models = ['Linear Regression', 'Polynomial Regression', 'SVR']

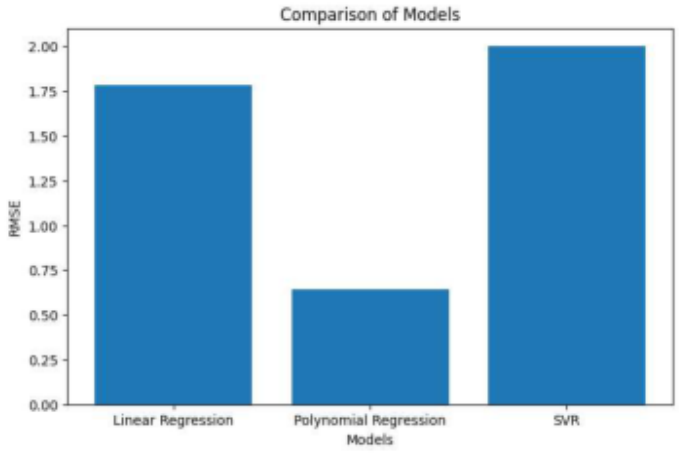
rmse_values = [
    rmse_linear,
    rmse_poly,
    rmse_svr
]

plt.figure(figsize=(8, 5))

plt.bar(models, rmse_values)

plt.xlabel("Models")
plt.ylabel("RMSE")
plt.title("Comparison of Models")

plt.show()
```




```
from sklearn.cluster import KMeans

X_cluster = df[['TV', 'Radio', 'Newspaper']]
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
df['Cluster'] = kmeans.fit_predict(X_cluster)

print(df.head())
```

```
   Unnamed: 0     TV  Radio  Newspaper  Sales  Cluster
0           1  230.1   37.8       69.2   22.1        2
1           2   44.5   39.3       45.1   10.4        1
2           3   17.2   45.9       69.3    9.3        1
3           4  151.5   41.3       58.5   18.5        0
4           5  180.8   10.8       58.4   12.9        0
```

```
sns.scatterplot(
    data=df,
    x='TV',
    y='Sales',
    hue='Cluster'
)
plt.xlabel("TV Advertising")
plt.ylabel("Sales")
plt.title("K-Means Clustering: TV vs Sales")

plt.show()
```



RESULT

The Advertising data was explored, a train-test split was created, and regression models generated predicted sales values, as shown in the notebook outputs.

EXPERIMENT 3(b): HOUSE PRICE DATA — REGRESSION

AIM

To preprocess the house-price dataset and prepare it for regression-based price prediction.

QUESTION / TASK

Load house_price.csv. Inspect the dataset shape, data types, descriptive statistics, and missing values; remove duplicate rows; encode the categorical columns using the mappings in the notebook; separate price as the target; and split the data into training and testing sets.

CODE AND OUTPUT


```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
from sklearn.cluster import KMeans
from google.colab import files

uploaded =  files.upload()
```

```
Choose Files No file chosen    Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving house price.csv to house price (1).csv
```

```
df = pd.read_csv("house_price.csv")
print(df.head())
```

```
      price  area  bedrooms  bathrooms  stories mainroad guestroom basement  \
0  13300000  7420         4          2        3      yes        no       no
1  12250000  8960         4          4        4      yes        no       no
2  12250000  9960         3          2        2      yes        no      yes
3  12215000  7500         4          2        2      yes        no      yes
4  11410000  7420         4          1        2      yes       yes      yes

  hotwaterheating airconditioning  parking prefarea furnishingstatus
0              no             yes        2      yes        furnished
1              no             yes        3       no        furnished
2              no              no        2      yes   semi-furnished
3              no             yes        3      yes        furnished
4              no             yes        2       no        furnished
```

```
print(df.shape)
print(df.info())
print(df.describe())
```

```
(545, 13)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 545 entries, 0 to 544
Data columns (total 13 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   price             545 non-null    int64
 1   area              545 non-null    int64
 2   bedrooms          545 non-null    int64
 3   bathrooms         545 non-null    int64
 4   stories           545 non-null    int64
 5   mainroad          545 non-null    object
 6   guestroom         545 non-null    object
 7   basement          545 non-null    object
 8   hotwaterheating   545 non-null    object
 9   airconditioning   545 non-null    object
 10  parking           545 non-null    int64
 11  prefarea          545 non-null    object
 12  furnishingstatus  545 non-null    object
dtypes: int64(6), object(7)
memory usage: 55.5+ KB
None
              price          area    bedrooms   bathrooms     stories  \
count  5.450000e+02    545.000000  545.000000  545.000000  545.000000
mean   4.766729e+06   5150.541284    2.965138    1.286239    1.805505
std    1.870440e+06   2170.141023    0.738064    0.502470    0.867492
min    1.750000e+06   1650.000000    1.000000    1.000000    1.000000
25%    3.430000e+06   3600.000000    2.000000    1.000000    1.000000
50%    4.340000e+06   4600.000000    3.000000    1.000000    2.000000
75%    5.740000e+06   6360.000000    3.000000    2.000000    2.000000
max    1.330000e+07  16200.000000    6.000000    4.000000    4.000000

          parking
count  545.000000
mean     0.693578
std      0.861586
min      0.000000
25%      0.000000
50%      0.000000
75%      1.000000
max      3.000000
```

```
print(df.isnull().sum())
```

```
price             0
area              0
bedrooms          0
bathrooms         0
stories           0
mainroad          0
guestroom         0
basement          0
hotwaterheating   0
airconditioning   0
parking           0
prefarea          0
furnishingstatus  0
dtype: int64
```

```
df = df.drop_duplicates()
```

```
df["mainroad"] = df["mainroad"].map({"yes": 1, "no": 0})
df["guestroom"] = df["guestroom"].map({"yes": 1, "no": 0})
df["basement"] = df["basement"].map({"yes": 1, "no": 0})
df["hotwaterheating"] = df["hotwaterheating"].map({"yes": 1, "no": 0})
df["airconditioning"] = df["airconditioning"].map({"yes": 1, "no": 0})
df["prefarea"] = df["prefarea"].map({"yes": 1, "no": 0})
```

```
df["furnishingstatus"] = df["furnishingstatus"].map({
    "furnished": 2,
    "semi-furnished": 1,
    "unfurnished": 0
})
```

```
print(df.head())
print(df.isnull().sum())
```

```
      price  area  bedrooms  bathrooms  stories  mainroad  guestroom  \
0  13300000  7420         4          2        3         1          0
1  12250000  8960         4          4        4         1          0
2  12250000  9960         3          2        2         1          0
3  12215000  7500         4          2        2         1          0
4  11410000  7420         4          1        2         1          1

   basement  hotwaterheating  airconditioning  parking  prefarea  \
0         0                0                1        2         1
1         0                0                1        3         0
2         1                0                0        2         1
3         1                0                1        3         1
4         1                0                1        2         0

   furnishingstatus
0                 2
1                 2
2                 1
3                 2
4                 2
price             0
area              0
bedrooms          0
bathrooms         0
stories           0
mainroad          0
guestroom         0
basement          0
hotwaterheating   0
airconditioning   0
parking           0
prefarea          0
furnishingstatus  0
dtype: int64
```

```
X = df.drop("price", axis=1)
y = df["price"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
```

```
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
linear_model = LinearRegression()
linear_model.fit(X_train_scaled, y_train)
y_pred_linear = linear_model.predict(X_test_scaled)
```

```
poly = PolynomialFeatures(degree=2)
X_train_poly = poly.fit_transform(X_train_scaled)
X_test_poly = poly.transform(X_test_scaled)
poly_model = LinearRegression()
poly_model.fit(X_train_poly, y_train)
y_pred_poly = poly_model.predict(X_test_poly)
```

```
svr_model = SVR(kernel="rbf")
svr_model.fit(X_train_scaled, y_train)
y_pred_svr = svr_model.predict(X_test_scaled)
```

```
linear_mae = mean_absolute_error(y_test, y_pred_linear)
linear_mse = mean_squared_error(y_test, y_pred_linear)
linear_rmse = np.sqrt(linear_mse)

poly_mae = mean_absolute_error(y_test, y_pred_poly)
poly_mse = mean_squared_error(y_test, y_pred_poly)
poly_rmse = np.sqrt(poly_mse)

svr_mae = mean_absolute_error(y_test, y_pred_svr)
svr_mse = mean_squared_error(y_test, y_pred_svr)
svr_rmse = np.sqrt(svr_mse)

print("Linear Regression")
print("MAE:", linear_mae)
print("MSE:", linear_mse)
print("RMSE:", linear_rmse)

print("\nPolynomial Regression")
print("MAE:", poly_mae)
print("MSE:", poly_mse)
print("RMSE:", poly_rmse)

print("\nSVR")
print("MAE:", svr_mae)
print("MSE:", svr_mse)
print("RMSE:", svr_rmse)
```

```
Linear Regression
MAE: 979679.6912959908
MSE: 1771751116594.841
RMSE: 1331071.4167895128

Polynomial Regression
MAE: 1001599.0627238948
MSE: 1760866098118.9014
RMSE: 1326976.2990842064

SVR
MAE: 1763889.476081326
MSE: 5567937467895.903
RMSE: 2359647.74233272
```

```
X_cluster = df.drop("price", axis=1)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_cluster)

kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

df["Cluster"] = kmeans.fit_predict(X_scaled)

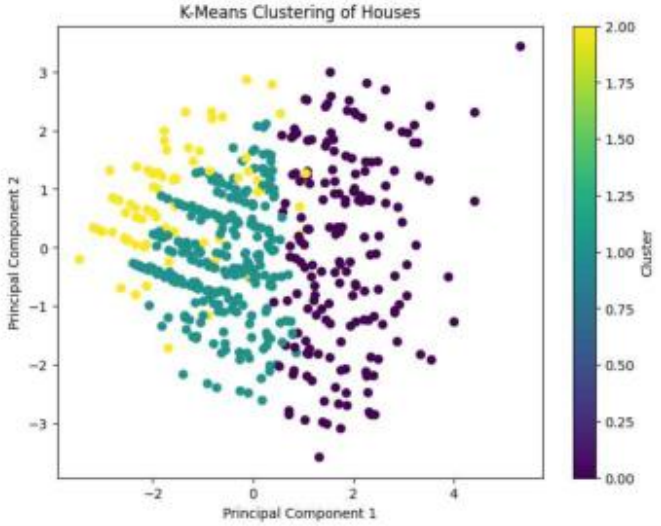
print(df[["price", "Cluster"]].head())
```

```
      price  Cluster
0  13300000        0
1  12250000        0
2  12250000        0
3  12215000        0
4  11410000        0
```

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
plt.figure(figsize=(8, 6))
plt.scatter(
    X_pca[:, 0],
    X_pca[:, 1],
    c=df["Cluster"],
    cmap="viridis"
)

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("K-Means Clustering of Houses")
plt.colorbar(label="Cluster")
plt.show()
```



exp 4

```
import pandas as pd
import re
import nltk

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
nltk.download("punkt")
nltk.download("stopwords")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
from google.colab import files
uploaded =  files.upload()
```


RESULT

The 545-row, 13-column housing dataset was inspected. The categorical fields were mapped to numeric values, the target price was separated from the features, and the data was split for model development.

EXPERIMENT 4: TEXT PREPROCESSING AND SIMILARITY

AIM

To clean review text and retrieve reviews similar to a query using TF-IDF and cosine similarity.

QUESTION / TASK

Load the Amazon fine-food reviews dataset, select the Text column, remove empty rows, normalize and clean text, remove stop words, create TF-IDF vectors, and calculate cosine similarity between a query and the reviews. Display the top matching reviews.

CODE AND OUTPUT


```
import pandas as pd
import re
import nltk

from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
True
```

```
url = "https://raw.githubusercontent.com/pplonski/datasets-for-start/refs/heads/master/amazon-fine-food-reviews/amazon_fine_food
df = pd.read_csv(url)
print(df.shape)
```

```
(10000, 10)
```

```
df = df[["Text"]]
df = df.dropna()
print(df.shape)
```

```
(10000, 1)
```

```
stop_words = set(stopwords.words("english"))
print(list(stop_words)[:20])
```

```
['do', 'how', "it'll", 'there', 'above', 'he', "they've", "couldn't", 'does', 'same', 'weren', 'couldn', "weren't", 's', 'mustn'
```

```
text = df["Text"].iloc[0]
text = text.lower()
print(text)
```

```
i have bought several of the vitality canned dog food products and have found them all to be of good quality. the product looks
```

```
text = re.sub(r"[^a-zA-Z\s]", "", text)
print(text)
```

```
i have bought several of the vitality canned dog food products and have found them all to be of good quality the product looks m
```

```
tokens = word_tokenize(text)
print(tokens)
```

```
['i', 'have', 'bought', 'several', 'of', 'the', 'vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'found', 'them',
```

```
tokens = [word for word in tokens if word not in stop_words]
print(tokens)
```

```
['bought', 'several', 'vitality', 'canned', 'dog', 'food', 'products', 'found', 'good', 'quality', 'product', 'looks', 'like', '
```

```
cleaned_text = " ".join(tokens)
print(cleaned_text)
```

```
bought several vitality canned dog food products found good quality product looks like stew processed meat smells better labrado
```

```
def preprocess(text):
    text = text.lower()
    text = re.sub(r"[^a-zA-Z\s]", "", text)
```

```
    tokens = word_tokenize(text)
    tokens = [word for word in tokens if word not in stop_words]
    return " ".join(tokens)
```

```
df["Cleaned_Text"] = df["Text"].apply(preprocess)
print(df[["Text", "Cleaned_Text"]].head())
```

```
                                                Text  \
0  I have bought several of the Vitality canned d...
1  Product arrived labeled as Jumbo Salted Peanut...
2  This is a confection that has been around a fe...
3  If you are looking for the secret ingredient i...
4  Great taffy at a great price.  There was a wid...

                                        Cleaned_Text
0  bought several vitality canned dog food produc...
1  product arrived labeled jumbo salted peanutsth...
2  confection around centuries light pillowy citr...
3  looking secret ingredient robitussin believe f...
4  great taffy great price wide assortment yummy ...
```

```
vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(df["Cleaned_Text"])
print(tfidf_matrix.shape)
```

```
(10000, 23530)
```

```
query = "great product with fast shipping"
print(query)
cleaned_query = preprocess(query)
print(cleaned_query)
```

```
great product with fast shipping
great product fast shipping
```

```
query_vector = vectorizer.transform([cleaned_query])
print(query_vector.shape)
```

```
(1, 23530)
```

```
similarity = cosine_similarity(query_vector, tfidf_matrix)
print(similarity.shape)
scores = similarity[0]
```

```
(1, 10000)
```

```
top_indices = scores.argsort()[-5:][::-1]
for i in top_indices:
    print("Similarity Score:", scores[i])
    print("Review:")
    print(df.iloc[i]["Text"])
    print("-" * 50)
```

```
Similarity Score: 0.5023833725407261
Review:
Enjoyed the product and they also provided very fast shipping.  I'm about out and need to order more.
--------------------------------------------------
Similarity Score: 0.4439167027723155
Review:
The tea is good and fresh. We enjoy it. The shipping is fast and cost is reasonable.<br />I recommend this product.
--------------------------------------------------
Similarity Score: 0.40573810454134535
Review:
My daughter lives in Hawaii and sent me some great coffee from there. I have a Keurig and  was not sure how to use the new coffe
--------------------------------------------------
Similarity Score: 0.4030724437516561
Review:
The energy drink is a great product. The shipping price is crazy tho. I paid more for the shipping then both products together,
--------------------------------------------------
Similarity Score: 0.39034766965263545
Review:
Fast shipping, items were packaged nicely and are as described. Also, very tasty :) Highly recommend this product to anyone look
--------------------------------------------------
```

```
query = "disappointed"
cleaned_query = preprocess(query)
```

```
query_vector = vectorizer.transform([cleaned_query])
similarity = cosine_similarity(query_vector, tfidf_matrix)
scores = similarity[0]
top_indices = scores.argsort()[-5:][::-1]
for i in top_indices:
    print("Similarity Score:", scores[i])
    print("Review:")
    print(df.iloc[i]["Text"])
    print("-" * 50)
```

```
Similarity Score: 0.48502899088339163
Review:
I am a bit disappointed.  The flavor was not what I wanted or expected.
--------------------------------------------------
Similarity Score: 0.3568832512700832
Review:
The product is very good. Way too expensive and at almost 5.00 a box to get pancakes or biscuits I was disappointed
--------------------------------------------------
Similarity Score: 0.3208255222381509
Review:
Disappointed.  The big boxes had a very different flavor than the smaller boxes.  I am not sure why but not good.
--------------------------------------------------
Similarity Score: 0.311352652121441 36
Review:
Just plain nasty!!! This item tasted like cardboard, very watered down and no flavor to it at all.  Very disappointed
--------------------------------------------------
Similarity Score: 0.30885315082050266
Review:
this stuff really works, i love it and cant get enough.  It tastes amazing  Highly recommend trying it, you wont be disappointed
--------------------------------------------------
```

```
for i in top_indices:
    print("Original Review:")
    print(df.iloc[i]["Text"])

    print("\nCleaned Review:")
    print(df.iloc[i]["Cleaned_Text"])

    print("\nSimilarity Score:", scores[i])
    print("-" * 60)
```

```
Original Review:
I am a bit disappointed.  The flavor was not what I wanted or expected.

Cleaned Review:
bit disappointed flavor wanted expected

Similarity Score: 0.48502899088339163
------------------------------------------------------------
Original Review:
The product is very good. Way too expensive and at almost 5.00 a box to get pancakes or biscuits I was disappointed

Cleaned Review:
product good way expensive almost box get pancakes biscuits disappointed

Similarity Score: 0.3568832512700832
------------------------------------------------------------
Original Review:
Disappointed.  The big boxes had a very different flavor than the smaller boxes.  I am not sure why but not good.

Cleaned Review:
disappointed big boxes different flavor smaller boxes sure good

Similarity Score: 0.3208255222381509
------------------------------------------------------------
Original Review:
Just plain nasty!!! This item tasted like cardboard, very watered down and no flavor to it at all.  Very disappointed

Cleaned Review:
plain nasty item tasted like cardboard watered flavor disappointed

Similarity Score: 0.311352652121441 36
------------------------------------------------------------
Original Review:
this stuff really works, i love it and cant get enough.  It tastes amazing  Highly recommend trying it, you wont be disappointed

Cleaned Review:
stuff really works love cant get enough tastes amazing highly recommend trying wont disappointed

Similarity Score: 0.30885315082050266
------------------------------------------------------------
```

RESULT

The review text was preprocessed, converted into a TF-IDF matrix, and compared with sample queries using cosine similarity. The highest-scoring reviews were displayed.

EXPERIMENT 5: NLP USING spaCy

AIM

To perform basic natural language processing on Amazon product reviews using spaCy.

QUESTION / TASK

Load the Amazon fine-food reviews dataset and select the Text column. Remove missing and duplicate rows, load the spaCy English model, and display tokens, lemmas, and part-of-speech tags for sample reviews.

CODE AND OUTPUT


```
!pip install spacy
```

```
Requirement already satisfied: spacy in /usr/local/lib/python3.13/dist-packages (3.8.16)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.12)
Requirement already satisfied: spacy-loggers<2.0.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.5)
Requirement already satisfied: murmurhash<1.1.0,>=0.28.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.15)
Requirement already satisfied: cymem<2.1.0,>=2.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.13)
Requirement already satisfied: preshed<3.1.0,>=3.0.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.0.13)
Requirement already satisfied: thinc<8.4.0,>=8.3.12 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.3.13)
Requirement already satisfied: wasabi<1.2.0,>=0.9.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.1.3)
Requirement already satisfied: srsly<3.0.0,>=2.5.3 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.5.4)
Requirement already satisfied: catalogue<2.1.0,>=2.0.6 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.0.10)
Requirement already satisfied: weasel<2.8.0,>=1.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.0.0)
Requirement already satisfied: confection<2.0.0,>=1.3.2 in /usr/local/lib/python3.13/dist-packages (from spacy) (1.3.3)
Requirement already satisfied: typer<1.0.0,>=0.3.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (0.27.2)
Requirement already satisfied: click<9.0.0,>=8.2.1 in /usr/local/lib/python3.13/dist-packages (from spacy) (8.5.0)
Requirement already satisfied: tqdm<5.0.0,>=4.38.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (4.67.3)
Requirement already satisfied: numpy>=1.19.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.1.3)
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.32.4)
Requirement already satisfied: pydantic<3.0.0,>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (2.13.5)
Requirement already satisfied: jinja2 in /usr/local/lib/python3.13/dist-packages (from spacy) (3.1.6)
Requirement already satisfied: setuptools in /usr/local/lib/python3.13/dist-packages (from spacy) (80.10.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from spacy) (26.3)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->s
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0->sp
Requirement already satisfied: typing-extensions>=4.14.1 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic<3.0.0,>=2.0.0-
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spacy) (3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spac
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.13/dist-packages (from requests<3.0.0,>=2.13.0->spac
Requirement already satisfied: blis<1.4.0,>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from thinc<8.4.0,>=8.3.12->spacy)
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (
Requirement already satisfied: rich>=13.8.0 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy) (13.9.4
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.13/dist-packages (from typer<1.0.0,>=0.3.0->spacy)
Requirement already satisfied: cloudpathlib>=0.7.0 in /usr/local/lib/python3.13/dist-packages (from weasel<2.8.0,>=1.0.0->spacy)
Requirement already satisfied: smart-open>=5.2.1 in /usr/local/lib/python3.13/dist-packages (from weasel<2.8.0,>=1.0.0->spacy) (
Requirement already satisfied: httpx>=0.24.0 in /usr/local/lib/python3.13/dist-packages (from weasel<2.8.0,>=1.0.0->spacy) (0.28
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.13/dist-packages (from jinja2->spacy) (3.0.3)
Requirement already satisfied: anyio in /usr/local/lib/python3.13/dist-packages (from httpx>=0.24.0->weasel<2.8.0,>=1.0.0->spacy
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx>=0.24.0->weasel<2.8.0,>=1.0.
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx>=0.24.0->weasel<2
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer<1.0.0,
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.13/dist-packages (from rich>=13.8.0->typer<1.0.
Requirement already satisfied: wrapt in /usr/local/lib/python3.13/dist-packages (from smart-open>=5.2.1->weasel<2.8.0,>=1.0.0->s
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.13/dist-packages (from markdown-it-py>=2.2.0->rich>=13.8.0->
```

```
import pandas as pd
import spacy
```

```
url = "https://raw.githubusercontent.com/pplonski/datasets-for-start/refs/heads/master/amazon-fine-food-reviews/amazon_fine_food
df = pd.read_csv(url)
print(df.head())
print(df.shape)
```

```
   Id   ProductId          UserId                      ProfileName  \
0   1  B001E4KFG0  A3SGXH7AUHU8GW                       delmartian
1   2  B00813GRG4  A1D87F6ZCVE5NK                           dll pa
2   3  B000LQOCH0   ABXLMWJIXXAIN  Natalia Corres "Natalia Corres"
3   4  B000UA0QIQ  A395BORC6FGVXV                             Karl
4   5  B006K2ZZ7K  A1UQRSCLF8GW1T    Michael D. Bigham "M. Wassir"

   HelpfulnessNumerator  HelpfulnessDenominator  Score        Time  \
0                     1                       1      5  1303862400
1                     0                       0      1  1346976000
2                     1                       1      4  1219017600
3                     3                       3      2  1307923200
4                     0                       0      5  1350777600

                Summary                                               Text
0  Good Quality Dog Food  I have bought several of the Vitality canned d...
1      Not as Advertised  Product arrived labeled as Jumbo Salted Peanut...
2  "Delight" says it all  This is a confection that has been around a fe...
3         Cough Medicine  If you are looking for the secret ingredient i...
4            Great taffy  Great taffy at a great price.  There was a wid...
(10000, 10)
```

```
df = df[["Text"]]
df = df.dropna()
df = df.drop_duplicates()
df = df.head(10000)
print(df.shape)
```

```
(9513, 1)
```

```
!python -m spacy download en_core_web_sm
```

```
Collecting en-core-web-sm==3.8.0
  Downloading https://github.com/explosion/spacy-models/releases/download/en_core_web_sm-3.8.0/en_core_web_sm-3.8.0-py3-none-any
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 89.5 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.
```

```
nlp = spacy.load("en_core_web_sm")
```

```
for text in df["Text"].head(5):
    doc = nlp(text)
    print("Original Review:")
    print(text)
    print("\nTokens:")
    print([token.text for token in doc])
    print("\nLemmas:")
    print([token.lemma_ for token in doc])
    print("\nPOS Tags:")
    print([token.pos_ for token in doc])
    print("-" * 80)
```

```
Original Review:
I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The product look

Tokens:
['I', 'have', 'bought', 'several', 'of', 'the', 'Vitality', 'canned', 'dog', 'food', 'products', 'and', 'have', 'found', 'them

Lemmas:
['I', 'have', 'buy', 'several', 'of', 'the', 'Vitality', 'can', 'dog', 'food', 'product', 'and', 'have', 'find', 'they', 'all'

POS Tags:
['PRON', 'AUX', 'VERB', 'ADJ', 'ADP', 'DET', 'PROPN', 'VERB', 'NOUN', 'NOUN', 'NOUN', 'CCONJ', 'AUX', 'VERB', 'PRON', 'PRON',
--------------------------------------------------------------------------------
Original Review:
Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if this was an erro

Tokens:
['Product', 'arrived', 'labeled', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanuts', 'were', 'actually', 'small', 's

Lemmas:
['product', 'arrived', 'label', 'as', 'Jumbo', 'Salted', 'Peanuts', '...', 'the', 'peanut', 'be', 'actually', 'small', 'sized'

POS Tags:
['NOUN', 'AUX', 'VERB', 'ADP', 'PROPN', 'PROPN', 'PROPN', 'PUNCT', 'DET', 'NOUN', 'AUX', 'ADV', 'ADJ', 'ADJ', 'VERB', 'PUNCT',
--------------------------------------------------------------------------------
Original Review:
This is a confection that has been around a few centuries.  It is a light, pillowy citrus gelatin with nuts - in this case Filb

Tokens:
['This', 'is', 'a', 'confection', 'that', 'has', 'been', 'around', 'a', 'few', 'centuries', '.', ' ', 'It', 'is', 'a', 'light'

Lemmas:
['this', 'be', 'a', 'confection', 'that', 'have', 'be', 'around', 'a', 'few', 'century', '.', ' ', 'it', 'be', 'a', 'light', '

POS Tags:
['PRON', 'AUX', 'DET', 'NOUN', 'PRON', 'AUX', 'AUX', 'ADP', 'DET', 'ADJ', 'NOUN', 'PUNCT', 'SPACE', 'PRON', 'AUX', 'DET', 'ADJ
--------------------------------------------------------------------------------
Original Review:
If you are looking for the secret ingredient in Robitussin I believe I have found it.  I got this in addition to the Root Beer

Tokens:
['If', 'you', 'are', 'looking', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have', 'found'

Lemmas:
['if', 'you', 'be', 'look', 'for', 'the', 'secret', 'ingredient', 'in', 'Robitussin', 'I', 'believe', 'I', 'have', 'find', 'it

POS Tags:
```

```
['SCONJ', 'PRON', 'AUX', 'VERB', 'ADP', 'DET', 'ADJ', 'NOUN', 'ADP', 'PROPN', 'PRON', 'VERB', 'PRON', 'AUX', 'VERB', 'PRON', 'P
--------------------------------------------------------------------------------
Original Review:
Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very quick.  If your a taffy lover, th

Tokens:
['Great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'There', 'was', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy', '

Lemmas:
['great', 'taffy', 'at', 'a', 'great', 'price', '.', ' ', 'there', 'be', 'a', 'wide', 'assortment', 'of', 'yummy', 'taffy', '.

--- -
```

```
for text in df["Text"].head(5):
    doc = nlp(text)
    nouns = [token.text for token in doc if token.pos_ == "NOUN"]
    verbs = [token.text for token in doc if token.pos_ == "VERB"]
    adjectives = [token.text for token in doc if token.pos_ == "ADJ"]
    adverbs = [token.text for token in doc if token.pos_ == "ADV"]
    print("Review:")
    print(text)
    print("Nouns:", nouns)
    print("Verbs:", verbs)
    print("Adjectives:", adjectives)
    print("Adverbs:", adverbs)
    print("-" * 60)
```

```
Review:
I have bought several of the Vitality canned dog food products and have found them all to be of good quality. The product looks
Nouns: ['dog', 'food', 'products', 'quality', 'product', 'stew', 'meat', 'product']
Verbs: ['bought', 'canned', 'found', 'looks', 'processed', 'smells', 'appreciates']
Adjectives: ['several', 'good', 'finicky', 'most']
Adverbs: ['more', 'better', 'better']
------------------------------------------------------------
Review:
Product arrived labeled as Jumbo Salted Peanuts...the peanuts were actually small sized unsalted. Not sure if this was an error
Nouns: ['Product', 'peanuts', 'error', 'vendor', 'product']
Verbs: ['labeled', 'unsalted', 'intended', 'represent']
Adjectives: ['small', 'sized', 'sure']
Adverbs: ['actually']
------------------------------------------------------------
Review:
This is a confection that has been around a few centuries.  It is a light, pillowy citrus gelatin with nuts - in this case Filbe
Nouns: ['confection', 'centuries', 'citrus', 'nuts', 'case', 'Filberts', 'squares', 'sugar', 'chewy', 'treat', 'story', 'treat',
Verbs: ['gelatin', 'cut', 'coated', 'recommend', 'seduces', 'selling']
Adjectives: ['few', 'light', 'pillowy', 'tiny', 'powdered', 'tiny', 'mouthful', 'flavorful', 'yummy', 'familiar']
Adverbs: ['then', 'liberally', 'too', 'very', 'highly']
------------------------------------------------------------
Review:
If you are looking for the secret ingredient in Robitussin I believe I have found it.  I got this in addition to the Root Beer E
Nouns: ['ingredient', 'addition', 'cherry', 'soda', 'flavor']
Verbs: ['looking', 'believe', 'found', 'got', 'ordered', 'made']
Adjectives: ['secret', 'good', 'medicinal']
Adverbs: ['very']
------------------------------------------------------------
Review:
Great taffy at a great price.  There was a wide assortment of yummy taffy.  Delivery was very quick.  If your a taffy lover, thi
Nouns: ['taffy', 'price', 'assortment', 'taffy', 'lover', 'deal']
Verbs: ['was']
Adjectives: ['Great', 'great', 'wide', 'yummy', 'quick']
Adverbs: ['very']
------------------------------------------------------------
```

```
from collections import Counter
pos_counts = Counter()
for text in df["Text"]:
    doc = nlp(text)
    for token in doc:
        pos_counts[token.pos_] += 1
print(pos_counts)
print("Nouns:", pos_counts["NOUN"])
print("Verbs:", pos_counts["VERB"])
print("Adjectives:", pos_counts["ADJ"])
print("Adverbs:", pos_counts["ADV"])
```

```
Counter({'NOUN': 137968, 'PRON': 96814, 'PUNCT': 95543, 'VERB': 91231, 'ADJ': 68375, 'ADP': 66036, 'DET': 65366, 'AUX': 54355, '
Nouns: 137968
Verbs: 91231
Adjectives: 68375
Adverbs: 48257
```

```
entities = []
for text in df["Text"]:
    doc = nlp(text)

    for ent in doc.ents:
        if ent.label_ in ["PERSON", "ORG", "DATE", "MONEY", "QUANTITY", "PRODUCT"]:
            entities.append([text, ent.text, ent.label_])
entity_df = pd.DataFrame(
    entities,
    columns=["Review", "Entity", "Label"]
)
print(entity_df.head(20))
```

```
                                               Review  \
0   I have bought several of the Vitality canned d...
1   I have bought several of the Vitality canned d...
2   Product arrived labeled as Jumbo Salted Peanut...
3   This is a confection that has been around a fe...
4   This is a confection that has been around a fe...
5   This is a confection that has been around a fe...
6   This is a confection that has been around a fe...
7   This is a confection that has been around a fe...
8   This is a confection that has been around a fe...
9   Great taffy at a great price.  There was a wid...
10  I got a wild hair for taffy and ordered this f...
11  I got a wild hair for taffy and ordered this f...
12  This saltwater taffy had great flavors and was...
13  Right now I'm mostly just sprouting this so my...
14  I don't know if it's the cactus or the tequila...
15  My cats have been happily eating Felidae Plati...
16  My cats have been happily eating Felidae Plati...
17  good flavor! these came securely packed... the...
18  The Strawberry Twizzlers are my guilty pleasur...
19  The Strawberry Twizzlers are my guilty pleasur...

                              Entity     Label
0                           Vitality       ORG
1                         My Labrador    PERSON
2                        Jumbo Salted       ORG
3               around a few centuries      DATE
4               pillowy citrus gelatin       ORG
5                           C.S. Lewis'      ORG
6                               Edmund   PRODUCT
7                              Sisters    PERSON
8                                Witch       ORG
9                             Delivery    PERSON
10                           five pound  QUANTITY
11                       only two weeks      DATE
12                          Fralinger's      ORG
13                           Wheatgrass    PERSON
14  Tequila Picante Gourmet de Inclan       ORG
15                      Felidae Platinum      ORG
16                 more than two years      DATE
17                             Twizzlers      ORG
18            The Strawberry Twizzlers       ORG
19                          Six pounds  QUANTITY
```

```
def extract_information(review):
    doc = nlp(review)
    tokens = [token.text for token in doc]
    lemmas = [token.lemma_ for token in doc]
    pos_tags = [token.pos_ for token in doc]
    named_entities = [ent.text for ent in doc.ents]
    entity_labels = [ent.label_ for ent in doc.ents]
    return tokens, lemmas, pos_tags, named_entities, entity_labels
```

```
reviews = [
    "This product is excellent and I really enjoyed it.",
    "The delivery was late and the product was damaged.",
    "I bought this coffee for my family and everyone loved it."
]
```

```
for review in reviews:
    result = extract_information(review)

    print("Review:", review)
    print("Tokens:", result[0])
```


[This page contains visual content that could not be extracted as editable text.]

```
    print("Lemmas:", result[1])
    print("POS Tags:", result[2])
    print("Named Entities:", result[3])
    print("Entity Labels:", result[4])
    print("-" * 60)
```

```
Review: This product is excellent and I really enjoyed it.
Tokens: ['This', 'product', 'is', 'excellent', 'and', 'I', 'really', 'enjoyed', 'it', '.']
Lemmas: ['this', 'product', 'be', 'excellent', 'and', 'I', 'really', 'enjoy', 'it', '.']
POS Tags: ['DET', 'NOUN', 'AUX', 'ADJ', 'CCONJ', 'PRON', 'ADV', 'VERB', 'PRON', 'PUNCT']
Named Entities: []
Entity Labels: []
------------------------------------------------------------
Review: The delivery was late and the product was damaged.
Tokens: ['The', 'delivery', 'was', 'late', 'and', 'the', 'product', 'was', 'damaged', '.']
Lemmas: ['the', 'delivery', 'be', 'late', 'and', 'the', 'product', 'be', 'damage', '.']
POS Tags: ['DET', 'NOUN', 'AUX', 'ADJ', 'CCONJ', 'DET', 'NOUN', 'AUX', 'VERB', 'PUNCT']
Named Entities: []
Entity Labels: []
------------------------------------------------------------
Review: I bought this coffee for my family and everyone loved it.
Tokens: ['I', 'bought', 'this', 'coffee', 'for', 'my', 'family', 'and', 'everyone', 'loved', 'it', '.']
Lemmas: ['I', 'buy', 'this', 'coffee', 'for', 'my', 'family', 'and', 'everyone', 'love', 'it', '.']
POS Tags: ['PRON', 'VERB', 'DET', 'NOUN', 'SCONJ', 'PRON', 'NOUN', 'CCONJ', 'PRON', 'VERB', 'PRON', 'PUNCT']
Named Entities: []
Entity Labels: []
------------------------------------------------------------
```

```
results = []

for review in reviews:
    tokens, lemmas, pos_tags, named_entities, entity_labels = extract_information(review)

    results.append({
        "Review": review,
        "Tokens": tokens,
        "Lemmas": lemmas,
        "POS Tags": pos_tags,
        "Named Entities": named_entities,
        "Entity Labels": entity_labels
    })

result_df = pd.DataFrame(results)
print(result_df)
```

```
                                      Review  \
0  This product is excellent and I really enjoyed...
1  The delivery was late and the product was dama...
2  I bought this coffee for my family and everyon...

                                      Tokens  \
0  [This, product, is, excellent, and, I, really,...
1  [The, delivery, was, late, and, the, product, ...
2  [I, bought, this, coffee, for, my, family, and...

                                      Lemmas  \
0  [this, product, be, excellent, and, I, really,...
1  [the, delivery, be, late, and, the, product, b...
2  [I, buy, this, coffee, for, my, family, and, e...

                                    POS Tags Named Entities  \
0  [DET, NOUN, AUX, ADJ, CCONJ, PRON, ADV, VERB, ...              []
1  [DET, NOUN, AUX, ADJ, CCONJ, DET, NOUN, AUX, V...              []
2  [PRON, VERB, DET, NOUN, SCONJ, PRON, NOUN, CCO...              []

  Entity Labels
0            []
1            []
2            []
```

RESULT

The spaCy English model processed selected reviews. Tokens, lemmas, and POS tags were displayed, demonstrating basic text analysis.

EXPERIMENT 6(a): NETFLIX DATASET EXPLORATORY ANALYSIS

AIM

To explore the Netflix titles dataset and visualize the distribution and trends of its content.

QUESTION / TASK

Load the Netflix dataset and inspect its records, data types, and summary. Plot Movies versus TV Shows, rating distribution, top countries, titles by release year, titles added by year, movie-duration distribution, and the number of seasons for TV Shows, as included in the Colab PDF.

CODE AND OUTPUT


```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
url = "https://raw.githubusercontent.com/prasertcbs/basic-dataset/master/netflix_titles.csv"
df = pd.read_csv(url)
```

```
print(df.head())
print(df.tail())
print(df.shape)
print(df.columns)
print(df.info())
print(df.isnull().sum())
print(df.describe(include="all"))
```

```
date_added        11
release_year       0
rating            10
duration           0
listed_in          0
description        0
dtype: int64
             show_id   type        title                director  \
count   6.234000e+03   6234         6234                    4265
unique           NaN      2         6172                    3301
top              NaN  Movie  The Silence  Raúl Campos, Jan Suter
freq             NaN   4265            3                      18
mean    7.670368e+07    NaN          NaN                     NaN
std     1.094296e+07    NaN          NaN                     NaN
min     2.477470e+05    NaN          NaN                     NaN
25%     8.003580e+07    NaN          NaN                     NaN
50%     8.016337e+07    NaN          NaN                     NaN
75%     8.024489e+07    NaN          NaN                     NaN
max     8.123573e+07    NaN          NaN                     NaN

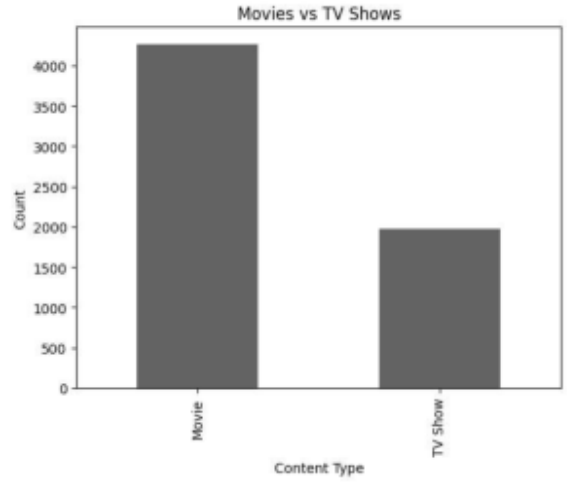
                       cast        country       date_added  release_year  \
count                  5664           5758             6223   6234.00000
unique                 5469            554             1524          NaN
top     David Attenborough  United States  January 1, 2020          NaN
freq                     18           2032              122          NaN
mean                    NaN            NaN              NaN   2013.35932
std                     NaN            NaN              NaN      8.81162
min                     NaN            NaN              NaN   1925.00000
25%                     NaN            NaN              NaN   2013.00000
50%                     NaN            NaN              NaN   2016.00000
75%                     NaN            NaN              NaN   2018.00000
max                     NaN            NaN              NaN   2020.00000

       rating  duration      listed_in  \
count    6224      6234           6234
unique     14       201            461
top     TV-MA  1 Season  Documentaries
freq     2027      1321            299
mean      NaN       NaN            NaN
std       NaN       NaN            NaN
min       NaN       NaN            NaN
25%       NaN       NaN            NaN
50%       NaN       NaN            NaN
75%       NaN       NaN            NaN
max       NaN       NaN            NaN

                                                description
count                                                6234
unique                                               6226
top     A surly septuagenarian gets another chance at ...
freq                                                    3
mean                                                  NaN
std                                                   NaN
min                                                   NaN
25%                                                   NaN
50%                                                   NaN
75%                                                   NaN
max                                                   NaN
```

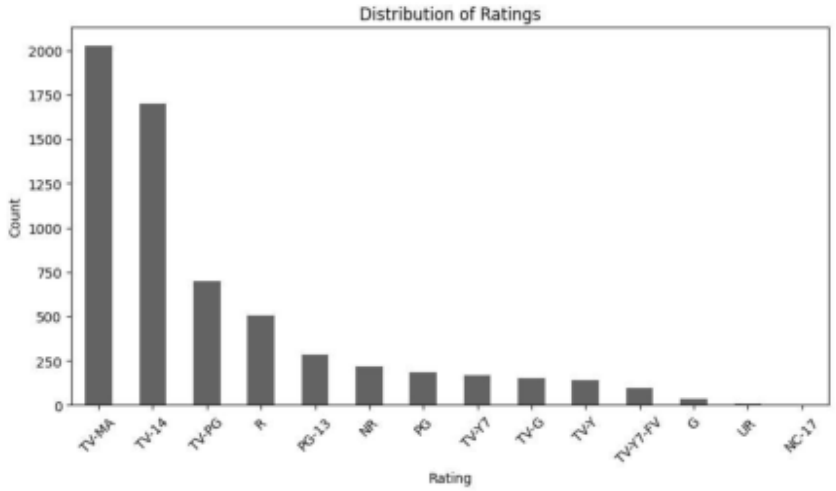
```
print(df["type"].value_counts())
df["type"].value_counts().plot(kind="bar")
plt.title("Movies vs TV Shows")
plt.xlabel("Content Type")
plt.ylabel("Count")
plt.show()
```

```
type
Movie      4265
TV Show    1969
Name: count, dtype: int64
```



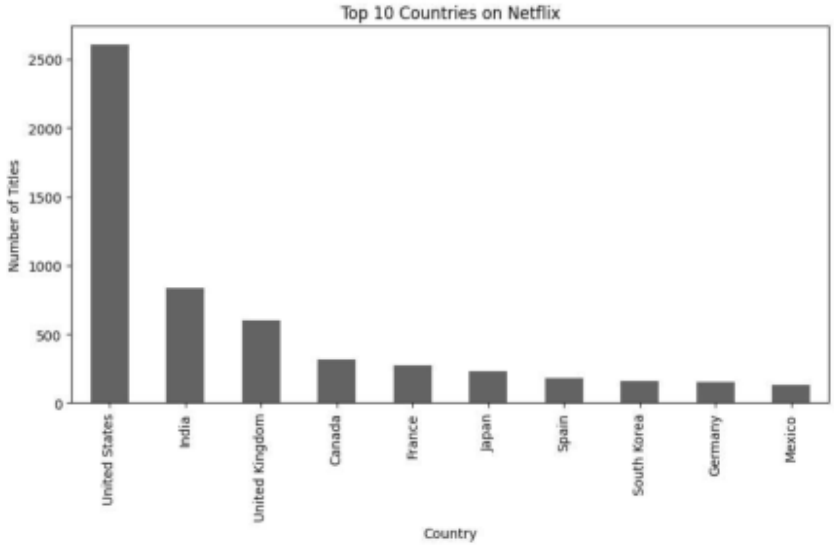

```
print(df["rating"].value_counts())
df["rating"].value_counts().plot(kind="bar", figsize=(10,5))
plt.title("Distribution of Ratings")
plt.xlabel("Rating")
plt.ylabel("Count")
plt.xticks(rotation=45)
plt.show()
```

```
rating
TV-MA       2027
TV-14       1698
TV-PG        701
R            508
PG-13        286
NR           218
PG           184
TV-Y7        169
TV-G         149
TV-Y         143
TV-Y7-FV      95
G             37
UR             7
NC-17          2
Name: count, dtype: int64
```



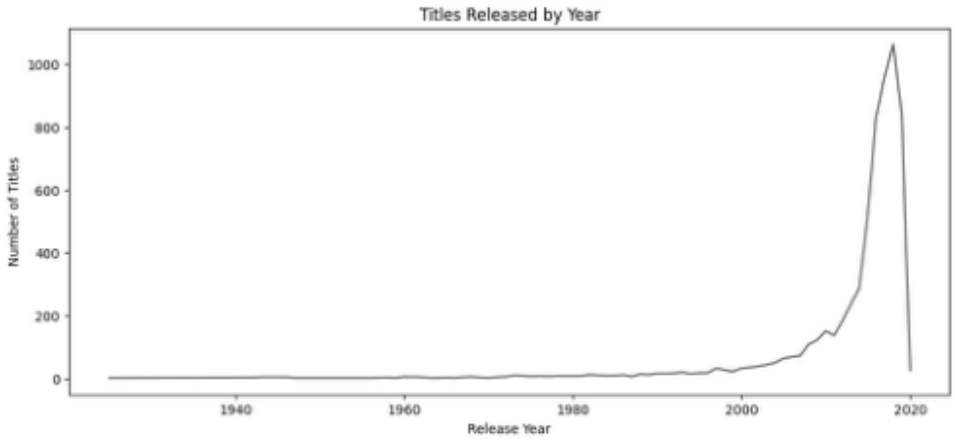

```
countries = df["country"].dropna().str.split(", ").explode()
print(countries.value_counts().head(10))
countries.value_counts().head(10).plot(kind="bar", figsize=(10,5))
plt.title("Top 10 Countries on Netflix")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.show()
```

```
country
United States     2609
India              838
United Kingdom     601
Canada             318
France             271
Japan              231
Spain              178
South Korea        162
Germany            151
Mexico             129
Name: count, dtype: int64
```



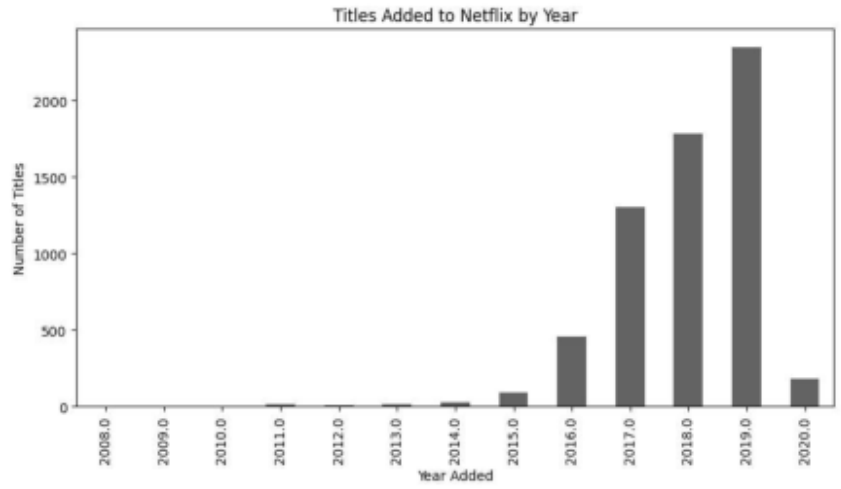

```
print(df["release_year"].value_counts().head(10))
df["release_year"].value_counts().sort_index().plot(figsize=(12,5))
plt.title("Titles Released by Year")
plt.xlabel("Release Year")
plt.ylabel("Number of Titles")
plt.show()
```

```
release_year
2018    1063
2017     959
2019     843
2016     838
2015     517
2014     288
2013     237
2012     183
2010     149
2011     136
Name: count, dtype: int64
```

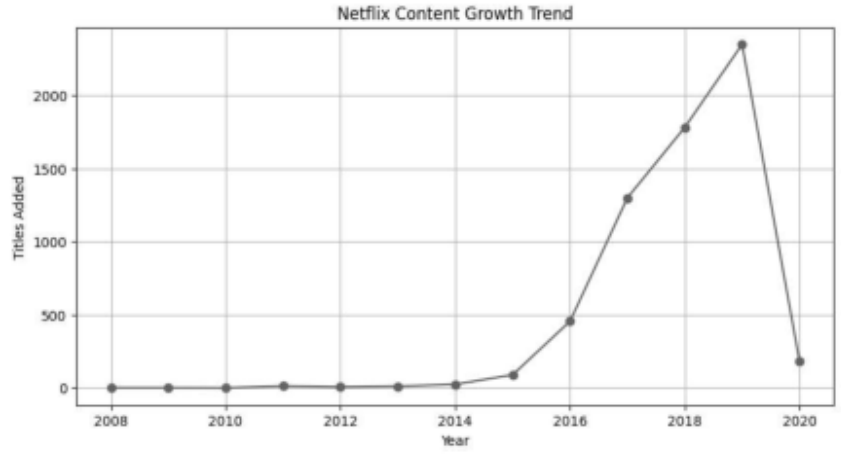



```
df["date_added"] = pd.to_datetime(df["date_added"].str.strip())
print(df["date_added"].dt.year.value_counts().sort_index())
df["date_added"].dt.year.value_counts().sort_index().plot(kind="bar", figsize=(10,5))
plt.title("Titles Added to Netflix by Year")
plt.xlabel("Year Added")
plt.ylabel("Number of Titles")
plt.show()
```

```
date_added
2008.0       2
2009.0       2
2010.0       1
2011.0      13
2012.0       7
2013.0      12
2014.0      25
2015.0      90
2016.0     456
2017.0    1300
2018.0    1782
2019.0    2349
2020.0     184
Name: count, dtype: int64
```




```
yearly = df["date_added"].dt.year.value_counts().sort_index()
yearly.plot(kind="line", marker="o", figsize=(10,5))
plt.title("Netflix Content Growth Trend")
plt.xlabel("Year")
plt.ylabel("Titles Added")
plt.grid()
plt.show()
```

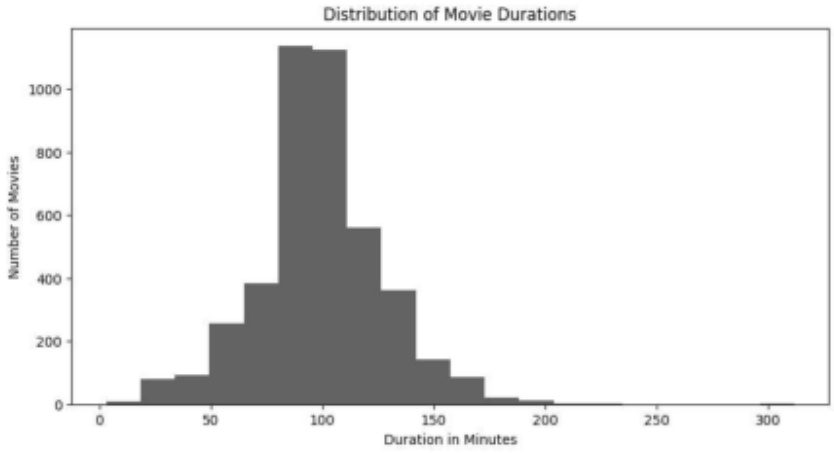


```
movies = df[df["type"] == "Movie"].copy()
movies["duration_minutes"] = pd.to_numeric(
    movies["duration"].str.replace(" min", "", regex=False),
    errors="coerce"
)

print(movies["duration_minutes"].describe())

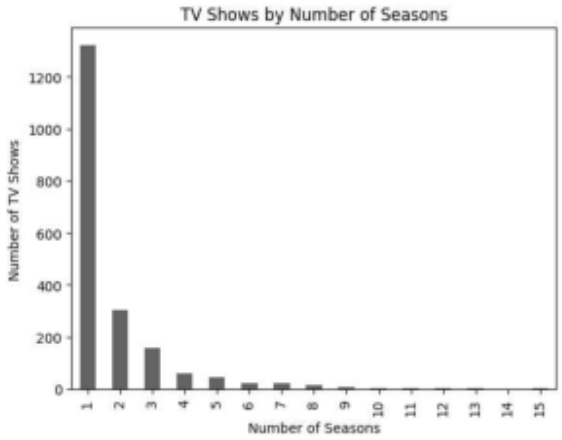
movies["duration_minutes"].dropna().plot(
    kind="hist", bins=20, figsize=(10,5)
)
plt.title("Distribution of Movie Durations")
plt.xlabel("Duration in Minutes")
plt.ylabel("Number of Movies")
plt.show()
```

```
count    4265.000000
mean       99.100821
std        28.074857
min         3.000000
25%        86.000000
50%        98.000000
75%       115.000000
max       312.000000
Name: duration_minutes, dtype: float64
```



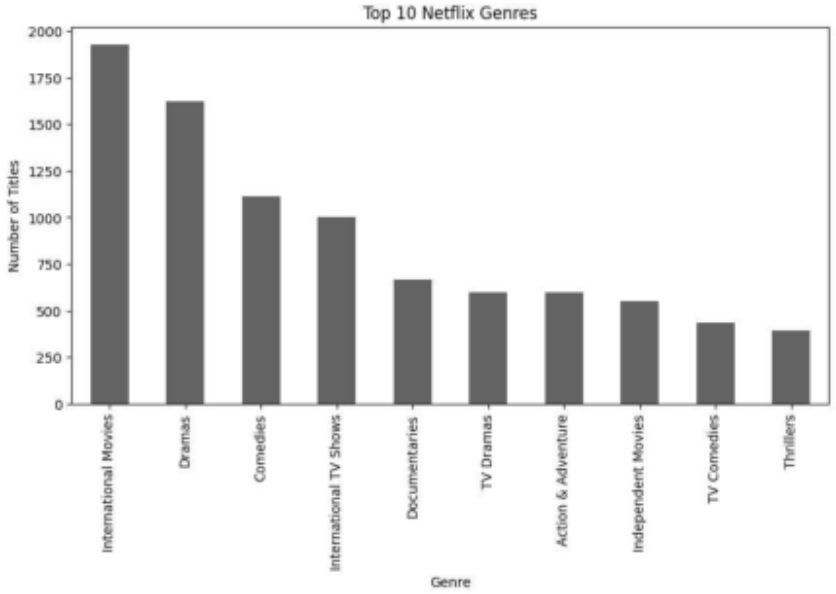

```
shows = df[df["type"] == "TV Show"].copy()
shows["seasons"] = pd.to_numeric(
    shows["duration"].str.replace(" Seasons", "", regex=False)
    .str.replace(" Season", "", regex=False),
    errors="coerce"
)
print(shows["seasons"].value_counts().sort_index())
shows["seasons"].value_counts().sort_index().plot(kind="bar")
plt.title("TV Shows by Number of Seasons")
plt.xlabel("Number of Seasons")
plt.ylabel("Number of TV Shows")
plt.show()
```

```
seasons
1     1321
2      304
3      158
4       61
5       46
6       22
7       21
8       16
9        7
10       3
11       3
12       2
13       2
14       1
15       2
Name: count, dtype: int64
```



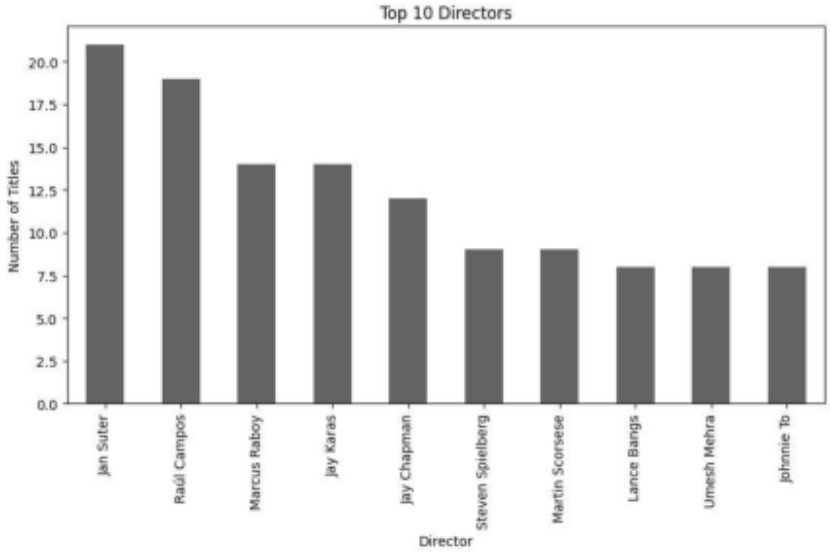

```
genres = df["listed_in"].dropna().str.split(", ").explode()
print(genres.value_counts().head(10))
genres.value_counts().head(10).plot(kind="bar", figsize=(10,5))
plt.title("Top 10 Netflix Genres")
plt.xlabel("Genre")
plt.ylabel("Number of Titles")
plt.show()
```

```
listed_in
International Movies      1927
Dramas                    1623
Comedies                  1113
International TV Shows    1001
Documentaries              668
TV Dramas                  599
Action & Adventure         597
Independent Movies         552
TV Comedies                436
Thrillers                  392
Name: count, dtype: int64
```



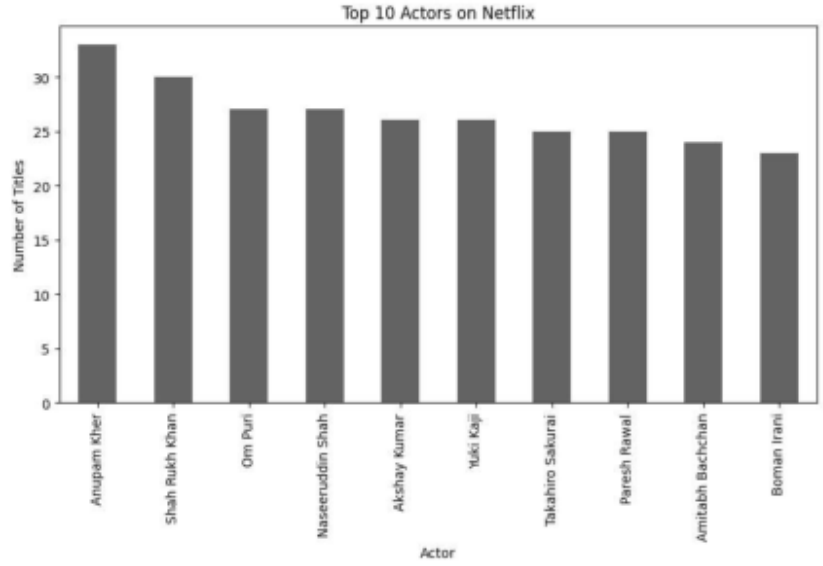

```
directors = df["director"].dropna().str.split(", ").explode()
print(directors.value_counts().head(10))
directors.value_counts().head(10).plot(kind="bar", figsize=(10,5))
plt.title("Top 10 Directors")
plt.xlabel("Director")
plt.ylabel("Number of Titles")
plt.show()
```

```
director
Jan Suter            21
Raúl Campos          19
Marcus Raboy         14
Jay Karas            14
Jay Chapman          12
Steven Spielberg      9
Martin Scorsese       9
Lance Bangs           8
Umesh Mehra           8
Johnnie To            8
Name: count, dtype: int64
```




```
actors = df["cast"].dropna().str.split(", ").explode()
print(actors.value_counts().head(10))
actors.value_counts().head(10).plot(kind="bar", figsize=(10,5))
plt.title("Top 10 Actors on Netflix")
plt.xlabel("Actor")
plt.ylabel("Number of Titles")
plt.show()
```

```
cast
Anupam Kher          33
Shah Rukh Khan       30
Om Puri              27
Naseeruddin Shah     27
Akshay Kumar         26
Yuki Kaji            26
Takahiro Sakurai     25
Paresh Rawal         25
Amitabh Bachchan     24
Boman Irani          23
Name: count, dtype: int64
```




```
!pip install wordcloud
```

```
Requirement already satisfied: wordcloud in /usr/local/lib/python3.13/dist-packages (1.9.6)
Requirement already satisfied: numpy>=1.19.3 in /usr/local/lib/python3.13/dist-packages (from wordcloud) (2.1.3)
Requirement already satisfied: pillow in /usr/local/lib/python3.13/dist-packages (from wordcloud) (11.3.0)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.13/dist-packages (from wordcloud) (3.10.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (1.4.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (4.66.1
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (1.5.1)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (26.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (3.3.3)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.13/dist-packages (from matplotlib->wordcloud) (2.9
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.7->matplotlib->wordc
```

```
from wordcloud import WordCloud
text = " ".join(df["description"].dropna())
wordcloud = WordCloud(
    width=1000,
    height=500,
    background_color="white"
).generate(text)
plt.figure(figsize=(12,6))
plt.imshow(wordcloud, interpolation="bilinear")
plt.axis("off")
plt.title("Common Words in Netflix Descriptions")
plt.show()
```




```
months = df["date_added"].dt.month_name().value_counts()
month_order = [
    "January", "February", "March", "April",
    "May", "June", "July", "August",
    "September", "October", "November", "December"
]
months = months.reindex(month_order)
print(months)
months.plot(kind="bar", figsize=(10,5))
plt.title("Netflix Content Added by Month")
plt.xlabel("Month")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```

```
date_added
January      610
February     378
March        551
April        447
May          428
June         393
July         474
```

```
country_type = df.dropna(subset=["country"]).copy()
country_type["country"] = country_type["country"].str.split(", ")
country_type = country_type.explode("country")
top_countries = country_type["country"].value_counts().head(10).index
data = country_type[country_type["country"].isin(top_countries)]
table = pd.crosstab(data["country"], data["type"])
print(table)
table.plot(kind="bar", figsize=(12,6))
plt.title("Movies vs TV Shows by Country")
plt.xlabel("Country")
plt.ylabel("Number of Titles")
plt.xticks(rotation=45)
plt.show()
```


RESULT

The Netflix data was summarized and visualized. The notebook outputs include plots for content type, ratings, countries, release-year and addition trends, movie durations, and TV-show seasons.

EXPERIMENT 6(b): AIRLINE PASSENGERS TIME-SERIES ANALYSIS

AIM

To analyze the monthly airline-passenger time series and apply ARIMA forecasting.

QUESTION / TASK

Upload and load the airline-passengers CSV with Month parsed as dates and set as the index. Display the first and last records, dataset shape and information, check missing values and duplicates, inspect descriptive statistics, and follow the time-series/ARIMA modelling and evaluation steps shown in the notebook.

CODE AND OUTPUT

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings("ignore")
```

```
from google.colab import files
import pandas as pd
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving airline-passengers.csv to airline-passengers.csv

```
df = pd.read_csv(
    'airline-passengers.csv',
    parse_dates=['Month'],
    index_col='Month'
)

print("Dataset uploaded and loaded successfully!")
display(df.head())
```

```
Dataset uploaded and loaded successfully!
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

```
print("First 10 records:")
display(df.head(10))
print("\nLast 10 records:")
display(df.tail(10))
print("\nDataset shape:")
print(df.shape)
print("\nDataset information:")
df.info()
```


[This page contains visual content that could not be extracted as editable text.]

```
First 10 records:
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121
1949-06-01	135
1949-07-01	148
1949-08-01	148
1949-09-01	136
1949-10-01	119

```
Last 10 records:
```

Month	Passengers
1960-03-01	419
1960-04-01	461
1960-05-01	472
1960-06-01	535
1960-07-01	622
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

```
Dataset shape:
(144, 1)

Dataset information:
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
```

```
print("Missing values:")
print(df.isnull().sum())

print("\nDuplicate rows:")
print(df.duplicated().sum())
```

```
Missing values:
Passengers    0
dtype: int64

Duplicate rows:
26
```

```
print("Statistical Summary:")
display(df.describe())
```


[This page contains visual content that could not be extracted as editable text.]

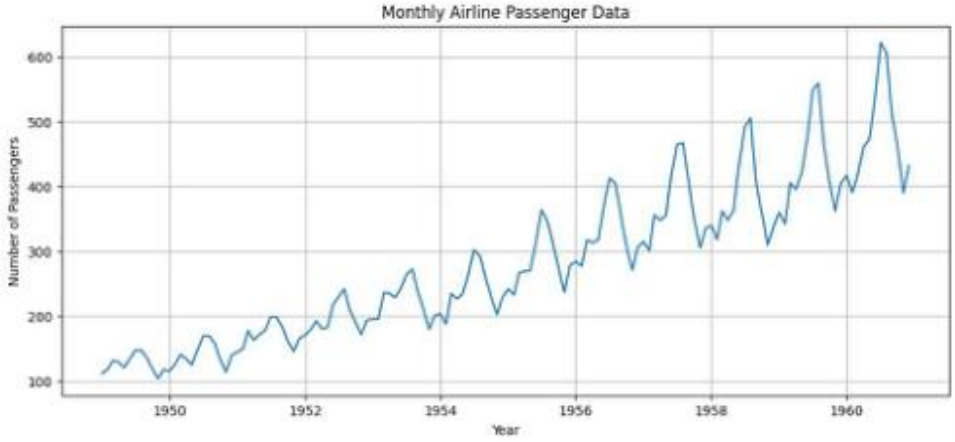
Statistical Summary:

	Passengers
count	144.000000
mean	280.298611
std	119.966317
min	104.000000
25%	180.000000
50%	265.500000
75%	360.500000
max	622.000000

```
plt.figure(figsize=(12, 5))
plt.plot(df.index, df['Passengers'])

plt.title("Monthly Airline Passenger Data")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.grid(True)

plt.show()
```




```
result = adfuller(df['Passengers'])
print("ADF Test Results")
print("--------------------------")
print("ADF Statistic:", result[0])
print("p-value:", result[1])
print("Number of Lags:", result[2])
print("Number of Observations:", result[3])

print("\nCritical Values:")
for key, value in result[4].items():
    print(f"{key}: {value}")

if result[1] <= 0.05:
    print("\nConclusion: The series is STATIONARY.")
else:
    print("\nConclusion: The series is NON-STATIONARY.")
```

```
ADF Test Results
--------------------------
ADF Statistic: 0.8153688792060498
p-value: 0.991888243437641
Number of Lags: 13
Number of Observations: 130
```


[This page contains visual content that could not be extracted as editable text.]

```
Critical Values:
1%: -3.4816817173418295
5%: -2.8840418343195267
10%: -2.578770059171598

Conclusion: The series is NON-STATIONARY.
```

```
df_diff = df.diff().dropna()

print("Differenced Data:")
display(df_diff.head(10))
```

```
Differenced Data:
```

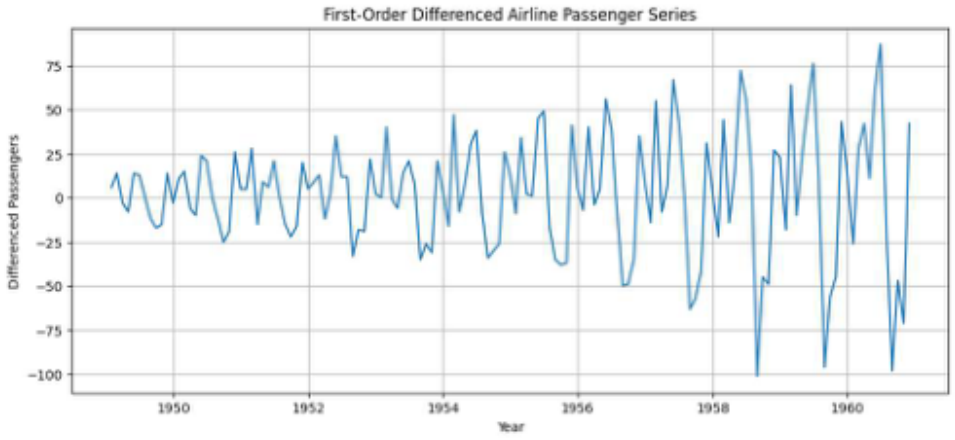
Month	Passengers
1949-02-01	6.0
1949-03-01	14.0
1949-04-01	-3.0
1949-05-01	-8.0
1949-06-01	14.0
1949-07-01	13.0
1949-08-01	0.0
1949-09-01	-12.0
1949-10-01	-17.0
1949-11-01	-15.0

```
plt.figure(figsize=(12, 5))

plt.plot(df_diff.index, df_diff['Passengers'])

plt.title("First-Order Differenced Airline Passenger Series")
plt.xlabel("Year")
plt.ylabel("Differenced Passengers")
plt.grid(True)

plt.show()
```




```
result_diff = adfuller(df_diff['Passengers'])

print("ADF Test After First Differencing")
print("----------------------------------")
```

```
print("ADF Statistic:", result_diff[0])
print("p-value:", result_diff[1])
print("Number of Lags:", result_diff[2])
print("Number of Observations:", result_diff[3])

print("\nCritical Values:")
for key, value in result_diff[4].items():
    print(f"{key}: {value}")

if result_diff[1] <= 0.05:
    print("\nConclusion: The differenced series is STATIONARY.")
else:
    print("\nConclusion: The differenced series is NON-STATIONARY.")
```

```
ADF Test After First Differencing
----------------------------------
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
Number of Lags: 12
Number of Observations: 130

Critical Values:
1%: -3.4816817173418295
5%: -2.8840418343195267
10%: -2.578770059171598

Conclusion: The differenced series is NON-STATIONARY.
```

```
if result_diff[1] <= 0.05:
    d = 1
else:
    d = 2

print("Selected differencing value (d):", d)
```

```
Selected differencing value (d): 2
```

```
train_size = int(len(df) * 0.80)

train = df['Passengers'].iloc[:train_size]
test = df['Passengers'].iloc[train_size:]

print("Training data size:", len(train))
print("Testing data size:", len(test))

print("\nTraining period:")
print(train.index[0], "to", train.index[-1])

print("\nTesting period:")
print(test.index[0], "to", test.index[-1])
```

```
Training data size: 115
Testing data size: 29

Training period:
1949-01-01 00:00:00 to 1958-07-01 00:00:00

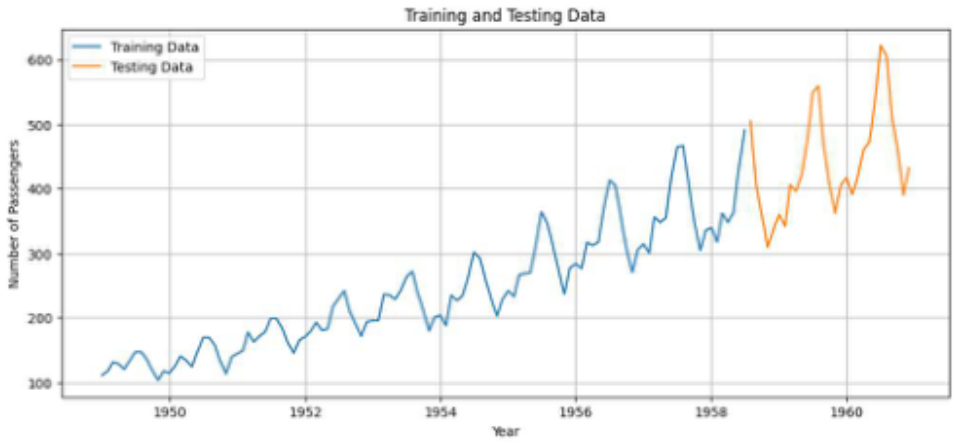
Testing period:
1958-08-01 00:00:00 to 1960-12-01 00:00:00
```

```
plt.figure(figsize=(12, 5))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Testing Data")

plt.title("Training and Testing Data")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)

plt.show()
```




```
p = 1
q = 1

model = ARIMA(train, order=(p, d, q))

model_fit = model.fit()

print(model_fit.summary())
```

```
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(1, 2, 1)   Log Likelihood                -529.588
Date:                Thu, 08 Oct 2026   AIC                           1065.177
Time:                        17:22:48   BIC                           1073.359
Sample:                    01-01-1949   HQIC                          1068.497
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.2919      0.103      2.839      0.005       0.090       0.493
ma.L1         -0.9997      4.323     -0.231      0.817      -9.472       7.473
sigma2       664.4540   2872.024      0.231      0.817   -4964.610    6293.518
===================================================================================
Ljung-Box (L1) (Q):                   0.30   Jarque-Bera (JB):                 3.26
Prob(Q):                              0.58   Prob(JB):                         0.20
Heteroskedasticity (H):               5.96   Skew:                             0.48
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.79
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

```
forecast = model_fit.forecast(steps=len(test))
forecast = pd.Series(
    forecast,
    index=test.index
)

print("Forecasted Passenger Numbers:")
display(forecast)
```

```
Forecasted Passenger Numbers:
```

Month	predicted_mean
1958-08-01	509.842168
1958-09-01	517.838000
1958-10-01	522.667796
1958-11-01	526.573429
1958-12-01	530.209300
1959-01-01	533.766428
1959-02-01	537.300571
1959-03-01	540.828004
1959-04-01	544.353479
1959-05-01	547.878382
1959-06-01	551.403118
1959-07-01	554.927806
1959-08-01	558.452479
1959-09-01	561.977149
1959-10-01	565.501817
1959-11-01	569.026484
1959-12-01	572.551152
1960-01-01	576.075819
1960-02-01	579.600487
1960-03-01	583.125154
1960-04-01	586.649822
1960-05-01	590.174489
1960-06-01	593.699157
1960-07-01	597.223824
1960-08-01	600.748492
1960-09-01	604.273159
1960-10-01	607.797827
1960-11-01	611.322494
1960-12-01	614.847162

```
comparison = pd.DataFrame({
    'Actual': test,
    'Predicted': forecast
})

display(comparison)
```


[This page contains visual content that could not be extracted as editable text.]

Month	Actual	Predicted
1958-08-01	505	509.842168
1958-09-01	404	517.838000
1958-10-01	359	522.667796
1958-11-01	310	526.573429
1958-12-01	337	530.209300
1959-01-01	360	533.766428
1959-02-01	342	537.300571
1959-03-01	406	540.828004
1959-04-01	396	544.353479
1959-05-01	420	547.878382
1959-06-01	472	551.403118
1959-07-01	548	554.927806
1959-08-01	559	558.452479
1959-09-01	463	561.977149
1959-10-01	407	565.501817
1959-11-01	362	569.026484
1959-12-01	405	572.551152
1960-01-01	417	576.075819
1960-02-01	391	579.600487

```
mse = mean_squared_error(test, forecast)

mae = mean_absolute_error(test, forecast)

rmse = np.sqrt(mse)

print("Model Evaluation Metrics")
print("------------------------")
print("MSE  :", mse)
print("MAE  :", mae)
print("RMSE :", rmse)
```

```
Model Evaluation Metrics
------------------------
MSE  : 20550.826985128582
MAE  : 126.99261583006495
RMSE : 143.35559608213984
```

```
plt.figure(figsize=(14, 6))

plt.plot(train.index, train, label="Training Data")
plt.plot(test.index, test, label="Actual Test Data")
plt.plot(forecast.index, forecast, label="ARIMA Prediction")

plt.title("ARIMA Forecasting of Airline Passenger Demand")
plt.xlabel("Year")
plt.ylabel("Number of Passengers")

plt.legend()
plt.grid(True)

plt.show()
```


RESULT

The monthly airline-passenger series was loaded and inspected. The notebook includes time-series analysis, ARIMA forecasting steps, and forecast evaluation outputs.

EXPERIMENT 7: MALL CUSTOMER SEGMENTATION USING K-MEANS

AIM

To examine the Mall Customers dataset and prepare income and spending-score features for customer clustering.

QUESTION / TASK

Upload Mall_Customers.csv. Display the first records, dataset shape, data types, missing and duplicate values, and descriptive statistics. Select Annual Income (k$) and Spending Score (1-100), standardize these features, and visualize their relationship before clustering.

CODE AND OUTPUT


```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving Mall_Customers.csv to Mall_Customers.csv

```
df = pd.read_csv("Mall_Customers.csv")
df.head()
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
df.tail()
print(df.shape)
print(df.dtypes)
```

```
(200, 5)
CustomerID                int64
Genre                    object
Age                       int64
Annual Income (k$)        int64
Spending Score (1-100)    int64
dtype: object
```

```
print("Missing values in each column:")
print(df.isnull().sum())
print("Number of duplicate records:", df.duplicated().sum())
```

```
Missing values in each column:
CustomerID              0
Genre                   0
Age                     0
Annual Income (k$)      0
Spending Score (1-100)  0
dtype: int64
Number of duplicate records: 0
```

```
df.describe()
```

	CustomerID	Age	Annual Income (k$)	Spending Score (1-100)
count	200.000000	200.000000	200.000000	200.000000
mean	100.500000	38.850000	60.560000	50.200000
std	57.879185	13.969007	26.264721	25.823522
min	1.000000	18.000000	15.000000	1.000000
25%	50.750000	28.750000	41.500000	34.750000
50%	100.500000	36.000000	61.500000	50.000000
75%	150.250000	49.000000	78.000000	73.000000
max	200.000000	70.000000	137.000000	99.000000

```
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print("Selected features:")
print(X.head())
```

```
Selected features:
   Annual Income (k$)  Spending Score (1-100)
```

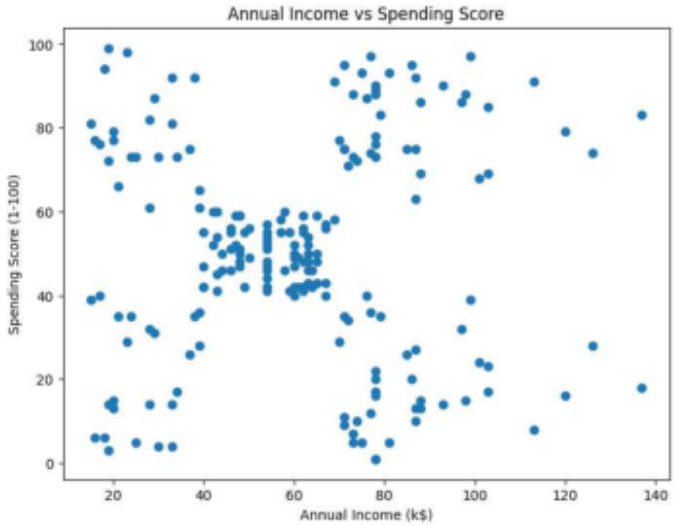
```
0              15                39
1              15                81
2              16                 6
3              16                77
4              17                40
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled = pd.DataFrame(
    X_scaled,
    columns=['Annual Income (k$)', 'Spending Score (1-100)']
)
print("Standardized features:")
print(X_scaled.head())
```

```
Standardized features:
   Annual Income (k$)  Spending Score (1-100)
0           -1.738999               -0.434801
1           -1.738999                1.195704
2           -1.700830               -1.715913
3           -1.700830                1.040418
4           -1.662660               -0.395980
```

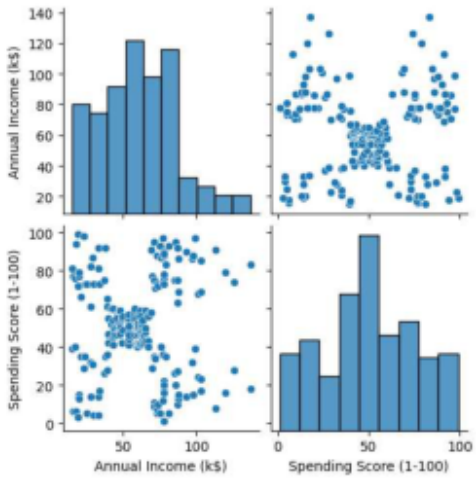
```
plt.figure(figsize=(8, 6))
plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)']
)

plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Annual Income vs Spending Score')
plt.show()
```




```
sns.pairplot(
    df[['Annual Income (k$)', 'Spending Score (1-100)']]
)

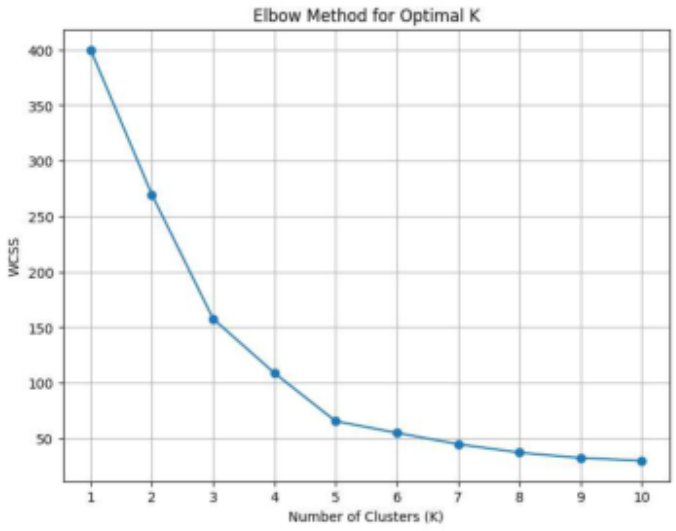
plt.show()
```




```
from sklearn.cluster import KMeans
wcss = []
for k in range(1, 11):
    kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=10
    )
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)
```

```
plt.figure(figsize=(8, 6))

plt.plot(
    range(1, 11),
    wcss,
    marker='o'
)
plt.xlabel('Number of Clusters (K)')
plt.ylabel('WCSS')
plt.title('Elbow Method for Optimal K')
plt.xticks(range(1, 11))
plt.grid(True)
plt.show()
```




```
optimal_k = 5
print("Optimal number of clusters:", optimal_k)
```

```
Optimal number of clusters: 5
```

```
kmeans = KMeans(
    n_clusters=optimal_k,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(X_scaled)

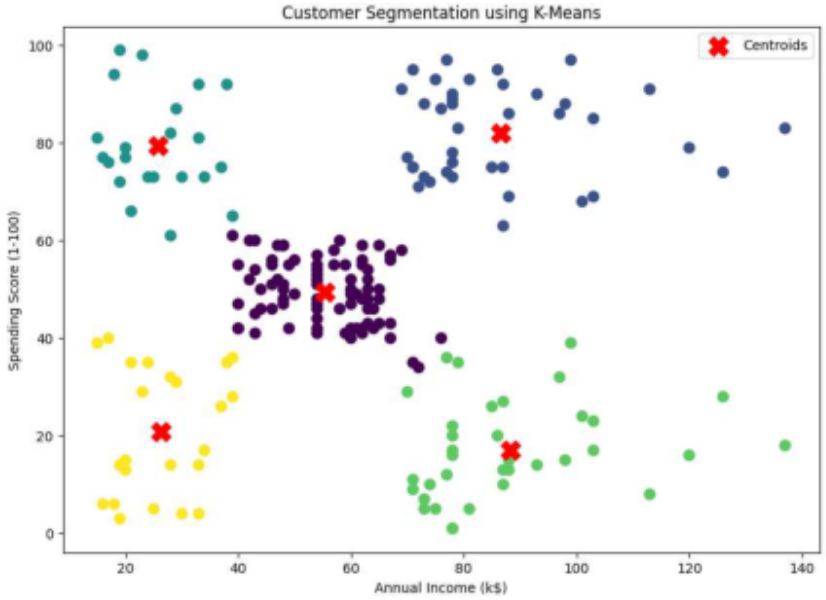
print("K-Means model trained successfully!")
df.head()
```

```
K-Means model trained successfully!
```

	CustomerID	Genre	Age	Annual Income (k$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	2
2	3	Female	20	16	6	4
3	4	Female	23	16	77	2
4	5	Female	31	17	40	4

```
plt.figure(figsize=(10, 7))
plt.scatter(
    df['Annual Income (k$)'],
    df['Spending Score (1-100)'],
    c=df['Cluster'],
    s=60,
    cmap='viridis'
)
centroids = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    s=200,
    marker='X',
    c='red',
    label='Centroids'
)
```

```
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.title('Customer Segmentation using K-Means')
plt.legend()
plt.show()
```




```
cluster_counts = df['Cluster'].value_counts().sort_index()
print("Number of customers in each cluster:")
print(cluster_counts)
```

```
Number of customers in each cluster:
Cluster
0    81
1    39
2    22
3    35
4    23
Name: count, dtype: int64
```

```
plt.figure(figsize=(8, 5))
cluster_counts.plot(
    kind='bar'
)

plt.xlabel('Cluster')
plt.ylabel('Number of Customers')
plt.title('Number of Customers in Each Cluster')

plt.show()
```



RESULT

The Mall Customers dataset was inspected. Annual Income and Spending Score were selected and standardized, and a scatter plot was generated to examine customer patterns for K-Means clustering.